付録 — 出てくるコマンドと設定の一覧

00〜08 と「参考 HTTPS にする場合」に出てくる 673 個のコマンド・オプション・SQL・コードの部品を、機能ごとに、意味と初めて出てくる所つきで引く

付録: 出てくるコマンドと設定の一覧（機能ごと）

この資料について

00〜08 と「参考 HTTPS にする場合」のコードの枠に出てくる、**PowerShell のコマンドとオプションと書き方・Windows のコマンド・XML の設定・SQL・psql・Python と Django・Rust と Topcoat の部品など 673 個**を、**36 の分類**に分けて、1 つずつ説明した一覧です。

- **本文との関係:** どのコマンドや部品も、本文の中で**初めて出てくるコードのすぐ下**に、「**▶ ここで初めて出てくるコード**」の箱で同じ説明を載せています。この一覧は、それを機能ごとに並べ直した物です。
- **使い方:** 「この `-StartupType` は何だっけ？」「`robocopy` の `/E` は？」と思ったら、下の「分類の一覧」から分類を選んで探してください。HTML 版（`onprem\html`）なら、`Ctrl` + `F` で、コマンドの名前で探すのがいちばん速いです。
- **オプションの書き方:** PowerShell のオプションは、`Get-Content -Tail` のように、**コマンドの名前と組にして**載せています。同じ `-Name` でも、コマンドによって意味が少しずつ違うからです。`-eq`・`-like`・`-join` などは、どのコマンドでも同じ意味の**演算子**なので、それだけで載せています。
- **「初めて出てくる所」の列:** そのコマンドや部品が最初に出てくる資料と章です（`04 7. アプリを Windows サービスにする` は、04 の 7 章）。そこで、実際にどう使われているかを見られます。
- **書き方の決まりも入っています:** PowerShell の `$_`・`|`・`"{0,-10}" -f`・`` ` ``（行の続き）・`@{ }`・日時の書式（`yyyyMMdd`）・正規表現（`^\s*#`）・WinSW と IIS の XML の設定・HTTP の見出し・SQL の `?`・psql の `\l` なども、1 つずつ説明しています。
- **やりたいことから探す時は:** 次の付録「逆引き」で、「サービスを止めたい」「古いログを消したい」のような、やりたいことからコマンドを探せます。本文に出てこない、合わせて覚えておくとよいコマンドも載せています。
- **名前の決まり:** 本番のサービスは `webapp-prod`、検証は `webapp-stg`、フォルダは `D:\webapp\…` です（04 の 4.1）。説明の中の名前も、これに合わせています。

> **ヒント:** 1 つのコマンドを覚えたら、その分類の表の、近くにある物も読んでみてください。例えば `PowerShell` の表には、サービスを見る（`Get-Service`）・止める（`Stop-Service`）・始める（`Start-Service`）・始め直す（`Restart-Service`）が並んでいます。一緒に覚えると、困った時に打てるコマンドがぐっと増えます。

分類の一覧

分類	数	分類	数
PowerShell	298	Option	4
日時の書式	6	エラー処理	3
正規表現	26	関数	4
コマンド	24	型とメソッド	6
設定ファイル（XML）	45	モジュール	4
HTTP のやり取り	8	所有権と借用	2
psql	26	スマートポインタ	1
SQL	75	時間	3
Python の道具	1	非同期	2
Node.js	2	クッキー	12
cargo のコマンド	3	認証	1
設定ファイル（TOML）	4	Topcoat	12
コメント	3	JavaScript・TypeScript	25
変数	2	TypeScript	9
文字列	1	React	1
Vec（並び）	2	Python	36
条件	1	Django	17
くり返し	1	Django REST framework	3

PowerShell

コード	意味と使い方	初めて出てくる所
`#`	`#` から行の終わりまでは「コメント」（人が読むためのメモ）で、PowerShell は無視します。コマンドの横に、何をするかを日本語で書いておけます。コピーしてそのまま貼り付けても、問題ありません。	00 0. はじめの一歩: コンピューターの中身

コード	意味と使い方	初めて出てくる所
`&`	`& "C:\Program Files\…\vswhere.exe"` の先頭の `&` は、「この文字（や変数）が指すプログラムを動かす」という印です。場所に空白が入っていて `"` で囲んだ時や、変数に入れた場所を動かす時に必要です。	00 0. はじめの一歩: コンピューターの中身
`Get-Content`	ファイルの中身を画面に出すコマンド（短い呼び名は `cat`）。`Get-Content memo.txt` で中身を表示します。日本語のファイルは `-Encoding utf8` を付けると文字化けしません。`-Tail 10` で最後の 10 行だけを出せます。	00 0. はじめの一歩: コンピューターの中身
`Get-Content -Encoding`	`-Encoding utf8` で、ファイルを UTF-8 として読みます。付けないと、Windows PowerShell 5.1 では日本語が化けることがあります。	00 0. はじめの一歩: コンピューターの中身
`Get-Content -TotalCount`	`Get-Content` で、ファイルの先頭から決めた行数だけを読むオプション。`-TotalCount 5` は先頭の 5 行。大きなファイルの中身を、少しだけのぞく時に使います。	00 0. はじめの一歩: コンピューターの中身
`Get-PSDrive`	PowerShell で使えるドライブ（`C:`・`D:` など）の一覧と、使っている量・空いている量を表示するコマンド。	00 0. はじめの一歩: コンピューターの中身
`Get-PSDrive -PSProvider`	`Get-PSDrive` で、種類を決めて取り出すオプション。`-PSProvider FileSystem` は、ディスクのドライブだけ（環境変数などの、ほかの種類を除く）。	00 0. はじめの一歩: コンピューターの中身
`Get-Service`	Windows のサービス（裏で動き続けるプログラム）の一覧と、動いているか（`Status`）を表示するコマンド。	00 3. OS
`Get-Service -Name`	`Get-Service` で、名前を決めて取り出すオプション。`-Name *postgres*` のように、`*` で「この文字を含む物」を探せます。	00 3. OS
`Select-Object`	結果のうち、見たい列だけを選ぶコマンド。`… | Select-Object Name, Length` のように、`|`（パイプ）で前のコマンドの結果を受け取って使います。`-First 5` で最初の 5 つだけにもできます。	00 3. OS
`|`	`|`（パイプ）は、左のコマンドの結果を、右のコマンドに流して渡す記号です。`Get-Service | Where-Object { … } | Select-Object Name` のように、「取り出す → しぼる → 選ぶ」をつないで書けます。キーボードでは `Shift` + `¥` で打ちます。	00 3. OS
`Get-NetIPConfiguration`	このパソコンのネットワークの設定（IP アドレス・ゲートウェイ・DNS サーバー）を表示するコマンド。	00 4. ネットワークの基礎

コード	意味と使い方	初めて出てくる所
`Resolve-DnsName`	名前（`webapp.example.local` など）を DNS に問い合わせて、IP アドレスを調べるコマンド。名前で開けない時、まずこれで「名前が引けているか」を確かめます。	00 4. ネットワークの基礎
`Test-NetConnection`	相手のパソコンにつながるかを確かめるコマンド。名前が引けるか・届くかを順に調べて表示します。	00 4. ネットワークの基礎
`Test-NetConnection -Port`	決めたポート番号につながるかを確かめるオプション。`-Port 80` の結果の `TcpTestSucceeded : True` なら、その窓口は開いています（False ならファイアウォールや、サービスが止まっていることを疑う）。	00 4. ネットワークの基礎
`Get-NetTCPConnection`	パソコンのネットワークの接続や、待ち受けているポート番号を一覧にするコマンド。「8080 番をだれが使っているか」を調べる時に使います。	00 4. ネットワークの基礎
`Get-NetTCPConnection -State`	`-State Listen` で、「待ち受けている」状態の物だけに絞ります（ほかに `Established` ＝つながっている、など）。	00 4. ネットワークの基礎
`Sort-Object`	受け取った物を、決めた項目で並べかえるコマンド。`Sort-Object LocalPort` は、ポート番号の小さい順。	00 4. ネットワークの基礎
`Invoke-WebRequest`	URL にリクエストを送って、答え（状態コード・本文）を受け取るコマンド。`(Invoke-WebRequest http://…/health -UseBasicParsing).StatusCode` で、状態コード（200 など）だけを取り出せます。	00 5. Web アプリの仕組み
`Invoke-WebRequest -UseBasicParsing`	答えの HTML を、Internet Explorer の部品を使わずに読むオプション。Windows Server で、IE の初めの設定が終わっていなくても動くように付けます。	00 5. Web アプリの仕組み
`.StatusCode`	`Invoke-WebRequest` の結果の `.StatusCode` は、HTTP の状態コード（`200` など）です。	00 5. Web アプリの仕組み
`[Security.Principal.WindowsPrincipal]`	`[Security.Principal.WindowsPrincipal]` は、「今の利用者が、どのグループにいるか」を調べるための .NET の型です。`.IsInRole(…)` で、管理者として動いているかを確かめられます。	01 3. PowerShell の基本
`[Security.Principal.WindowsBuiltInRole]`	`[Security.Principal.WindowsBuiltInRole]::Administrator` は、「Windows の管理者のグループ」を表す値です。	01 3. PowerShell の基本
`[型]::名前`	`[System.IO.File]::ReadAllBytes($path)` のように、`[型の名前]::名前` と書くと、.NET（PowerShell の土台）の型に付いている関数や値を、直接使えます。	01 3. PowerShell の基本

コード	意味と使い方	初めて出てくる所
`Get-Help`	コマンドの説明を表示するコマンド。`Get-Help Get-Service` で使い方、`-Examples` で例が見られます。	01 3. PowerShellの基本
`Get-Help -Examples`	`Get-Help` で、使い方の例だけを表示するオプション。初めてのコマンドは、まずこれで例を見ると早いです。	01 3. PowerShellの基本
`Get-Command`	コマンドがどこにあるか・何なのかを調べるコマンド。`(Get-Command rustc).Source` は、`rustc.exe` の場所を返します。入っているかを確かめる時に使います。	01 3. PowerShellの基本
`Get-History`	この窓で、これまでに打ったコマンドの一覧を表示するコマンド。	01 3. PowerShellの基本
`-eq`	PowerShellの「等しい」の比べ方（equal）。`$LASTEXITCODE -eq 0` は「終わった時の番号が0（成功）か」。PowerShellでは `==` ではなく `-eq` と書きます。大文字・小文字は区別しません（`"ABC" -eq "abc"` も `True`）。	01 3. PowerShellの基本
`Where-Object`	`|` で流れてきた物のうち、条件に合う物だけを残すコマンド。`{ }` の中の `$_` が「今の1つ」です。`Where-Object { $_.Length -gt 1000 }` のように使います。	01 3. PowerShellの基本
`'…'`	`'…'`（シングルクォート）で囲んだ文字は、書いたとおりの文字で、中の `$` や `` ` `` は特別な意味になりません。正規表現（`'^\s*#'`）や、そのまま渡したい文字に使います。中に `'` を入れる時は `''` と2つ書きます。	01 3. PowerShellの基本
`"…$変数…"`	`"…"`（ダブルクォート）で囲んだ文字の中の `$名前` は、変数の中身に置きかわります。`"こんにちは、$name さん"` → `こんにちは、太郎 さん`。そのまま `$` を出したい時は、`'…'`（シングルクォート）で囲みます。	01 3. PowerShellの基本
`$名前`	`$os = …` のように `$` で始まる名前は「変数」（値に付けた名前）です。`=` の右の結果を覚えておき、あとで `$os` と書けば使えます。名前の大文字・小文字は区別しません。	01 3. PowerShellの基本
`$env:名前`	`$env:GREETING` のように `$env:` を付けた名前は「環境変数」（プログラムに外から渡す設定）です。`$env:GREETING = "おはよう"` で設定すると、そのPowerShellの窓から動かすプログラム（`cargo run` など）が読めます。窓を閉じると消えます。	01 3. PowerShellの基本
`Get-Content -Tail`	`Get-Content` で、ファイルの最後から決めた行数だけを読むオプション。`-Tail 50` は最後の50行。ログの新しい所を見る時に使います。	01 3. PowerShellの基本

コード	意味と使い方	初めて出てくる所
`Get-Content -Wait`	ファイルの最後まで読んだあとも閉じずに待ち、新しく書き足された行を、その都度表示し続けるオプション。ログを見張る時に `-Tail 50 -Wait` と組み合わせます。止める時は `Ctrl` ＋ `C` です。	01 3. PowerShellの基本
`Select-String`	ファイルの中から、文字を含む行を探すコマンド。何行目に見つかったかも表示します。	01 3. PowerShellの基本
`Select-String -Path`	`-Path ファイル` で、探すファイルを指定します。`*.log` のように、いくつものファイルも指定できます。	01 3. PowerShellの基本
`Select-String -Pattern`	`-Pattern "文字"` で、探す文字を指定します。`"ERROR|WARN"` のように、`|` で「または」にもできます（正規表現）。	01 3. PowerShellの基本
`Select-Object -Last`	受け取った物の、最後から決めた数だけを取り出すオプション。`Select-Object -Last 20` は最後の 20 個。	01 3. PowerShellの基本
`Get-ChildItem`	フォルダの中の一覧を出すコマンド（短い呼び名は `ls`・`dir`）。`Get-ChildItem フォルダ` でそのフォルダの中身、`-Recurse` で中のフォルダまで全部を出します。	01 3. PowerShellの基本
`Get-ChildItem -Recurse`	`-Recurse` で、中のフォルダの中まで全部を一覧にします。	01 3. PowerShellの基本
`Get-ChildItem -File`	`Get-ChildItem` で、ファイルだけを取り出す（フォルダを除く）オプション。	01 3. PowerShellの基本
`Get-ChildItem -ErrorAction`	エラーが起きた時の動きを決めるオプション。`-ErrorAction SilentlyContinue` は、読めないフォルダ（権限が無い所）があっても、エラーを出さずに続けます。	01 3. PowerShellの基本
`Sort-Object -Descending`	大きい順（新しい順）に並べるオプション。`Sort-Object Length -Descending` は、大きいファイルから順に並べます。	01 3. PowerShellの基本
`Select-Object -First`	`-First 1` で、結果の最初の 1 つだけを取り出します。`-Last 3` なら最後の 3 つです。	01 3. PowerShellの基本
`$_`	`$_` は、`|`（パイプ）で流れてきた物の「今の 1 つ」を表す、特別な変数です。`Where-Object { $_.Name -like 'a*' }` は、「流れてきた 1 つずつの `Name` が a で始まる物だけ」。`catch { … }` の中では、起きたエラーを表します。	01 3. PowerShellの基本
`[int]`	`[int]` は「整数」という型です。`[int]($_.Length / 1MB)` のように値の前に書くと、整数に変えます（小数は四捨五入に近い形で丸めます）。	01 3. PowerShellの基本

コード	意味と使い方	初めて出てくる所
`;`	`;` は、1 行にコマンドを 2 つ以上書く時の区切りです。`cargo fmt --check; $LASTEXITCODE` は、左を動かしてから右を動かします（左が失敗しても、右は動きます）。	01 3. PowerShell の基本
`@{ … }`	`@{ キー = 値; … }` は「表（ハッシュテーブル）」を作る書き方です。`$h = @{ Name = "web"; Port = 8080 }` なら `$h.Port` で `8080`。`Select-Object` の中の `@{ Name = 'MB'; Expression = { … } }` は、「計算して作る列」の決まった形です。	01 3. PowerShell の基本
`.Length`	`$_.Length` は、ファイルなら大きさ（バイト）、文字なら文字数です。`/ 1MB` で MB に直せます（`1MB` は PowerShell が知っている 1048576 のことです）。	01 3. PowerShell の基本
`.\backup.ps1`	今いるフォルダの `backup.ps1`（この資料で作る、データベースとファイルの控えを取るスクリプト）を動かす書き方。PowerShell では、今いるフォルダのスクリプトも `.\` を付けないと動きません（うっかり同じ名前の物を動かさないための決まり）。	01 3. PowerShell の基本
`.\`	`.\target\debug\practice.exe` の `.\` は、「今いるフォルダの中の」という意味です。PowerShell は、安全のため、今いるフォルダのプログラムでも `.\` を付けないと動かしません。	01 3. PowerShell の基本
`Get-ExecutionPolicy`	今の実行ポリシー（スクリプトを動かしてよいかの決まり）を見るコマンド。`Restricted` はスクリプトを動かせない、`RemoteSigned` は自分で作った物は動かせる、という意味です。	01 3. PowerShell の基本
`Unblock-File`	インターネットやほかのパソコンから持ってきたファイルに付く「よそから来た」印を外すコマンド。印が付いたスクリプトは、決まりによって動かないことがあります。中身を確かめてから外します。	01 3. PowerShell の基本
`-join`	並びの中身を、区切り文字でつないで 1 つの文字列にする演算子。`$lines -join "`n"` は「改行でつなぐ」、`@("a","b") -join ","` は `"a,b"`。	01 3. PowerShell の基本
`Get-LocalUser`	このサーバーの中の利用者（ローカルユーザー）の一覧と、使えるか（`Enabled`）を表示するコマンド。	01 4. アカウントとグループ
`Get-LocalGroupMember`	このサーバーの中のグループに、だれが入っているかを表示するコマンド。	01 4. アカウントとグループ
`Get-LocalGroupMember -Group`	調べるグループの名前を決めるオプション。`-Group Administrators` は、管理者のグループ（入っている人は何でもできるので、少なくしておく）。	01 4. アカウントとグループ

コード	意味と使い方	初めて出てくる所
`Stop-Service`	Windows のサービスを止めるコマンド。管理者の PowerShell で動かします。	01 6. サービス
`Stop-Service -Name`	止めるサービスの名前を決めるオプション。`Stop-Service -Name webapp-stg`。	01 6. サービス
`Start-Service`	止まっている Windows のサービスを始めるコマンド。	01 6. サービス
`Start-Service -Name`	始めるサービスの名前を決めるオプション。	01 6. サービス
`Restart-Service`	Windows のサービスを、止めてから始め直すコマンド。設定ファイルを変えたあとに、読み直させる時に使います。	01 6. サービス
`Restart-Service -Name`	始め直すサービスの名前を決めるオプション。	01 6. サービス
`Set-Service`	Windows のサービスの設定（始まり方など）を変えるコマンド。	01 6. サービス
`Set-Service -Name`	設定を変えるサービスの名前を決めるオプション。	01 6. サービス
`Set-Service -StartupType`	サービスの始まり方を決めるオプション。`Automatic`（サーバーが起動したら自動で始まる）・`Manual`（手で始める）・`Disabled`（使えなくする）。	01 6. サービス
`Get-CimInstance`	Windows の中の情報（OS・CPU・サービス・ディスクなど）を問い合わせるコマンド。`Get-CimInstance Win32_OperatingSystem` は OS の情報、`Get-CimInstance Win32_Service` はサービスの一覧（自動で始まるかどうかも分かる）です。	01 6. サービス
`Get-CimInstance -Filter`	`Get-CimInstance` で、条件に合う物だけを取り出すオプション。`-Filter "Name='webapp-stg'"` は、名前が webapp-stg のサービスだけ。	01 6. サービス
`Get-WinEvent`	Windows のイベントログ（Windows やサービスが残す記録）を読むコマンド。	01 7. イベント ビューアー
`Get-WinEvent -FilterHashtable`	読むイベントの条件を、表（`@{ }`）で決めるオプション。`@{ LogName = 'System'; Level = 1, 2; StartTime = (Get-Date).AddDays(-1) }` は「System のログの、重大（1）とエラー（2）の、この 1 日の物」。`Id = 7031` のように、イベントの番号でも選べます。	01 7. イベント ビューアー
`Get-Date`	今の日時を返すコマンド。`(Get-Date).AddDays(-1)` は「今から 1 日前」、`(Get-Date).AddHours(-26)` は「26 時間前」です（`( )` で囲んでから、`.AddDays()` などのメソッドを呼びます）。	01 7. イベント ビューアー

コード	意味と使い方	初めて出てくる所
`Format-List`	受け取った物の項目を、1 行に 1 つずつ縦に並べて表示するコマンド。項目が多くて表（`Format-Table`）では切れてしまう時に使います。	01 7. イベント ビューアー
`Get-WinEvent -MaxEvents`	読むイベントの数の上限を決めるオプション（新しい物から）。`-MaxEvents 5` は最新の 5 件。	01 7. イベント ビューアー
`Get-WinEvent -ErrorAction`	エラーが起きた時の動きを決めるオプション。条件に合うイベントが 1 件も無いと、`Get-WinEvent` はエラーを出すので、`-ErrorAction SilentlyContinue` で黙らせます。	01 7. イベント ビューアー
`Get-Process`	動いているプログラム（プロセス）の一覧を出すコマンド。`Get-Process -Id 1234` で、番号 1234 のプロセスの名前などが分かります。	01 8. 重い・遅いを調べる
`-f`	書式（形）を決めて文字列を作る演算子。`"{0,-10} {1}" -f $name, $hash` は、`{0}` に 1 つ目、`{1}` に 2 つ目を入れます（`,-10` は「10 文字分の幅で左に寄せる」）。	01 8. 重い・遅いを調べる
`Get-Volume`	ディスク（ボリューム）の一覧と、大きさ・空いている量を表示するコマンド。	01 8. 重い・遅いを調べる
`Get-NetTCPConnection -LocalPort`	`-LocalPort 8080` で、このパソコン側のポート番号が 8080 の物だけに絞ります。	01 8. 重い・遅いを調べる
`"{0}" -f 値`	`"{0} と {1}" -f $a, $b` の `-f` は、左の文字の `{0}`・`{1}` に、右に並べた値を、0 番目から順に入れる書き方です（書式の演算子）。`"全体 {0:N1} GB" -f 15.6` → `全体 15.6 GB`。	01 8. 重い・遅いを調べる
`{0:N桁}`	`{0:N1}` の `N1` は、「3 けたごとに `,` を入れ、小数点以下を 1 けた」で表示する指定です。`N0` なら小数なし（`1,234`）。	01 8. 重い・遅いを調べる
`.WorkingSet64`	`Get-Process` の結果の `.WorkingSet64` は、そのプログラムが今使っているメモリの量（バイト）です。	01 8. 重い・遅いを調べる
`.TotalVisibleMemorySize`	`Win32_OperatingSystem` の `.TotalVisibleMemorySize` は、パソコンのメモリの全体の大きさです（単位は KB なので、`/1MB` で GB になります）。	01 8. 重い・遅いを調べる
`.FreePhysicalMemory`	`Win32_OperatingSystem` の `.FreePhysicalMemory` は、今空いているメモリの大きさです（単位は KB）。	01 8. 重い・遅いを調べる
`.SizeRemaining`	`Get-Volume` の結果の `.SizeRemaining` は、ドライブの空き容量（バイト）です。`.Size` は全体の大きさです。	01 8. 重い・遅いを調べる

コード	意味と使い方	初めて出てくる所
`.Size`	`Get-Volume` の結果の `.Size` は、ドライブ全体の大きさ（バイト）です。	01 8. 重い・遅いを調べる
`Get-ScheduledTask`	タスクスケジューラーに登録した「予定の仕事」の一覧を表示するコマンド。	01 9. タスク スケジューラ
`Get-ScheduledTask -TaskPath`	予定の仕事を入れたフォルダを決めるオプション。`-TaskPath "\webapp\"` は、この資料で作る仕事をまとめた `\webapp\` フォルダだけ。	01 9. タスク スケジューラ
`Get-ScheduledTaskInfo`	予定の仕事の、前に動いた時刻（`LastRunTime`）・結果（`LastTaskResult`。`0` なら成功）・次に動く時刻（`NextRunTime`）を表示するコマンド。	01 9. タスク スケジューラ
`Start-ScheduledTask`	予定の仕事を、時刻を待たずに今すぐ動かすコマンド。登録したあと、正しく動くかを確かめる時に使います。	01 9. タスク スケジューラ
`Start-ScheduledTask -TaskPath`	動かす予定の仕事のフォルダを決めるオプション。	01 9. タスク スケジューラ
`Start-ScheduledTask -TaskName`	動かす予定の仕事の名前を決めるオプション。	01 9. タスク スケジューラ
`Get-NetConnectionProfile`	ネットワークの種類（`DomainAuthenticated` ＝会社のドメイン・`Private`・`Public`）を表示するコマンド。ファイアウォールの決まりは、この種類ごとに効きます。	01 10. Windows Defender ファイアウォール
`Get-NetFirewallProfile`	ファイアウォールの、種類（Domain・Private・Public）ごとの設定と、有効か（`Enabled`）を表示するコマンド。	01 10. Windows Defender ファイアウォール
`Get-NetFirewallRule`	ファイアウォールの決まり（どの通信を通す・止めるか）の一覧を表示するコマンド。	01 10. Windows Defender ファイアウォール
`Get-NetFirewallRule -DisplayName`	表示名で決まりを探すオプション。`-DisplayName "webapp*"` は、名前が webapp で始まる決まりだけ。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule`	ファイアウォールに、新しい決まり（この通信を通す、など）を足すコマンド。管理者の PowerShell で動かします。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -DisplayName`	決まりの表示名を決めるオプション。あとで `Get-NetFirewallRule -DisplayName` で探せるように、分かりやすい名前を付けます。	01 10. Windows Defender ファイアウォール

コード	意味と使い方	初めて出てくる所
`New-NetFirewallRule -Direction`	通信の向きを決めるオプション。`Inbound` は「外からこのサーバーへ入ってくる通信」、`Outbound` は「このサーバーから出ていく通信」。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -Protocol`	通信の種類を決めるオプション。Web やデータベースは `TCP` です。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -LocalPort`	このサーバーの、どのポート（窓口の番号）への通信かを決めるオプション。`-LocalPort 80` は Web（HTTP）の窓口。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -RemoteAddress`	どこから来た通信を通すかを決めるオプション。`-RemoteAddress 10.0.0.0/8` のように、社内の住所の範囲だけに絞ります。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -Profile`	決まりを効かせるネットワークの種類を決めるオプション。`-Profile Domain` は、会社のドメインのネットワークの時だけ。	01 10. Windows Defender ファイアウォール
`New-NetFirewallRule -Action`	合った通信をどうするかを決めるオプション。`Allow` は通す、`Block` は止める。	01 10. Windows Defender ファイアウォール
`</code> | 行の最後の `` `（バッククォート。`Shift` ＋ `@` で打ちます）は、「次の行へ続く」という印です。長いコマンドを、何行かに分けて書けます。 `` ` `` の後ろに空白があると続かないので、注意します。`	01 10. Windows Defender ファイアウォール	
`Get-HotFix`	入っている Windows の更新プログラム（番号 `HotFixID` と、入れた日 `InstalledOn` ）の一覧を表示するコマンド。	01 11. Windows Update と再起動
`.LastBootUpTime`	`Win32_OperatingSystem` の `.LastBootUpTime` は、パソコンを最後に起動した日時です。	01 11. Windows Update と再起動
`Get-FileHash`	ファイルの中身から、ハッシュ（指紋のような長い英数字）を計算するコマンド。中身が 1 文字でも違えば、まったく違う値になります。ダウンロードしたファイルが壊れていないか確かめる時に使います。	01 12. ファイルを持ち込む

コード	意味と使い方	初めて出てくる所
`Get-FileHash -Algorithm`	ハッシュの計算のしかたを決めるオプション。`-Algorithm SHA256` は、配っている所が公開している SHA256 の値と比べて、ファイルが壊れていない・すりかえられていないかを確かめる時に使います。	01 12. ファイルを持ち込む
`+=`	`$x += 1` は `$x = $x + 1` の短い形です。並びに `$names += "山田"` と書くと、後ろに 1 つ足した並びになります。	02 2. ログインする
`Get-ComputerInfo`	このパソコンの情報（OS の名前と版・パソコンの名前・ドメインなど）を、まとめて表示するコマンド。	04 3. サーバーに入る・最初の確認
`Get-ComputerInfo -Property`	表示する項目を決めるオプション。`-Property OsName, OsVersion, CsName, CsDomain` は、OS の名前と版、パソコンの名前、ドメインだけ（全部だと、とても長い）。	04 3. サーバーに入る・最初の確認
`Format-Table`	結果を表の形に整えて表示するコマンド。	04 3. サーバーに入る・最初の確認
`Get-TimeZone`	このパソコンの時間帯（タイムゾーン）を表示するコマンド。日本なら `Tokyo Standard Time`（UTC+09:00）。	04 3. サーバーに入る・最初の確認
`Set-TimeZone`	このパソコンの時間帯を変えるコマンド。	04 3. サーバーに入る・最初の確認
`Set-TimeZone -Id`	変える先の時間帯の名前を決めるオプション。`-Id "Tokyo Standard Time"` は日本時間。	04 3. サーバーに入る・最初の確認
`foreach`	PowerShell のくり返し。`foreach ($line in Get-Content $envFile) { … }` は、ファイルの 1 行ずつを `$line` に入れて、中を動かします。	04 4. フォルダとアカウントの準備
`New-Item`	新しいファイルやフォルダを作るコマンド。`New-Item memo.txt` でファイル、`New-Item -ItemType Directory 名前` でフォルダを作ります。	04 4. フォルダとアカウントの準備
`New-Item -ItemType`	`-ItemType Directory` で、ファイルではなくフォルダを作ります（`File` ならファイル）。`New-Item -ItemType Directory src\bin` は、`src` の中に `bin` フォルダを作ります。	04 4. フォルダとアカウントの準備
`New-Item -Force`	`-Force` で、同じ名前のファイルがあっても上書きして作ります（無いと、エラーで止まります）。上書きするので、使う時は名前をよく確かめます。	04 4. フォルダとアカウントの準備
`New-Item -Path`	`-Path 場所` で、作るファイルやフォルダの場所（名前）を指定します。	04 4. フォルダとアカウントの準備

コード	意味と使い方	初めて出てくる所
`Out-Null`	コマンドの結果を、画面に出さずに捨てるコマンド。`… | Out-Null` のように後ろに付けると、表示が出なくなります。	04 4. フォルダとアカウントの準備
`@( … )`	`@( … )` は「並び（配列）」を作る書き方です。`@("a", "b")` で 2 つの並び、`@()` で空の並び。中のコマンドの結果が 1 つでも、必ず並びとして扱えます。	04 4. フォルダとアカウントの準備
`Copy-Item`	ファイルやフォルダをコピーするコマンド。`Copy-Item 元 先`。最後の `.` は「今いるフォルダへ」という意味です。	04 6. アプリを作る
`Copy-Item -Recurse`	`-Recurse` で、フォルダを中身ごと（中のフォルダも全部）コピーします。付けないと、フォルダの中身はコピーされません。	04 6. アプリを作る
`Compress-Archive`	ファイルやフォルダを、zip の 1 つのファイルにまとめるコマンド。	04 6. アプリを作る
`Compress-Archive -Path`	zip に入れる物を決めるオプション。`"$out\*"` は、そのフォルダの中身全部。	04 6. アプリを作る
`Compress-Archive -DestinationPath`	作る zip のファイルの場所と名前を決めるオプション。	04 6. アプリを作る
`Compress-Archive -Force`	同じ名前の zip がもうあれば、上書きするオプション。	04 6. アプリを作る
`*`	`'127*'` や `*.log` の `*` は、「何文字でも、何でも合う」という印（ワイルドカード）です。`-like '127*'` は「127 で始まる」、`-Filter *.log` は「名前が .log で終わる物」。1 文字だけなら `?` です。	04 6. アプリを作る
`Expand-Archive`	zip のファイルを、フォルダに広げる（解凍する）コマンド。	04 7. アプリを Windows サービスにする
`Expand-Archive -DestinationPath`	広げる先のフォルダを決めるオプション。	04 7. アプリを Windows サービスにする
`Remove-Item`	ファイルやフォルダを消すコマンド（短い呼び名は `rm`）。**ごみ箱に入らず、すぐ消えます。**フォルダを中身ごと消す時は `-Recurse` を付けます。消す前に、名前をよく確かめます。	04 7. アプリを Windows サービスにする
`Install-WindowsFeature`	Windows Server に、役割や機能（IIS など）を入れるコマンド。管理者の PowerShell で動かします。	04 8. 受付
`Install-WindowsFeature -Name`	入れる機能の名前を決めるオプション。`Web-Server` は IIS（Web サーバー）。	04 8. 受付

コード	意味と使い方	初めて出てくる所
`Install-WindowsFeature -IncludeManagementTools`	機能と一緒に、それを管理する道具（IIS マネージャーなど）も入れるオプション。	04 8. 受付
`Get-WindowsFeature`	Windows Server の役割や機能が、入っているか（`InstallState` が `Installed`）を表示するコマンド。	04 8. 受付
`Import-Module`	PowerShell に、コマンドのまとまり（モジュール）を読み込むコマンド。`Import-Module IISAdministration` で、`New-IISSite` などの IIS のコマンドが使えるようになります。	04 8. 受付
`Stop-IISSite`	IIS のサイトを止めるコマンド。	04 8. 受付
`Stop-IISSite -Name`	止めるサイトの名前を決めるオプション。`"Default Web Site"` は、IIS を入れた時にできる見本のサイト（使わないので止める）。	04 8. 受付
`Set-ItemProperty`	物（ファイル・レジストリ・IIS の設定など）の項目の値を変えるコマンド。`Set-ItemProperty "IIS:\Sites\Default Web Site" -Name serverAutoStart -Value $false` は「見本のサイトを、自動で始めない」。	04 8. 受付
`Set-ItemProperty -Name`	変える項目の名前を決めるオプション。	04 8. 受付
`Set-ItemProperty -Value`	変える先の値を決めるオプション。	04 8. 受付
`New-IISSite`	IIS に、新しいサイトを作るコマンド。	04 8. 受付
`New-IISSite -Name`	作るサイトの名前を決めるオプション。	04 8. 受付
`New-IISSite -PhysicalPath`	サイトの中身（設定ファイル `web.config` など）を置くフォルダを決めるオプション。	04 8. 受付
`New-IISSite -BindingInformation`	どの住所・ポート・名前で来た通信を、このサイトが受け取るかを決めるオプション。`"*:80:webapp.example.local"` は「どの住所でも・80 番で・名前が webapp.example.local の物」。同じ 80 番でも、名前で検証用と本番用を振り分けられます。	04 8. 受付
`Get-IISSite`	IIS のサイトの一覧と、動いているか・受け取る住所（バインド）を表示するコマンド。	04 8. 受付
`$false`	`$false` は「いいえ（偽）」を表す値です。`-Confirm:$false` は、「本当にしますか？」の確認を出さずに進める指定です。	04 8. 受付
`Get-NetFirewallPortFilter`	ファイアウォールの決まりの、ポート番号の条件の一覧を表示するコマンド。`| Get-NetFirewallRule` につなぐと、そのポートを使う決まりが分かります。	04 9. ファイアウォール

コード	意味と使い方	初めて出てくる所
`-in`	左の値が、右の並びの中にあるかの比べ方。`$_.LocalPort -in 80, 443` は「80 か 443 か」。	04 9. ファイアウォール
`{ … }`	`Where-Object { … }` や `ForEach-Object { … }` の `{ }` は「スクリプトブロック」（あとで動かす、命令のかたまり）です。流れてきた 1 つずつに対して、中の命令が動き、`$_` がその 1 つを表します。	04 9. ファイアウォール
`.LocalPort`	`.LocalPort` は、待ち受けやつながりの、このパソコン側のポート番号です。	04 9. ファイアウォール
`-notlike`	`-like` の逆で、ワイルドカードに合わないかの比べ方。`$_.DisplayName -notlike "webapp*"` は「webapp で始まらないか」。	04 9. ファイアウォール
`Disable-NetFirewallRule`	ファイアウォールの決まりを、消さずに使わない状態にするコマンド。あとで `Enable-NetFirewallRule` で戻せます。	04 9. ファイアウォール
`.DisplayName`	`.DisplayName` は、画面に出る名前です（サービスやファイアウォールの決まりの、人が読む名前）。	04 9. ファイアウォール
`Format-Table -AutoSize`	`-AutoSize` で、列の幅を中身に合わせて自動で決めます（長い値が `...` で切られにくくなります）。	04 9. ファイアウォール
`Resolve-DnsName -Type`	問い合わせる記録の種類を決めるオプション。`-Type MX` は、そのドメインのメールを受け取るサーバーを調べます（決めないと、IP アドレスの `A` を調べる）。	04 10. メール
`Send-MailMessage`	メールを送るコマンド。ここでは、メールの中継（コネクタ）が働くかを、アプリより先に確かめるために使います（古いコマンドで、新しく作る物には使わないよう案内されています）。	04 10. メール
`Send-MailMessage -SmtpServer`	メールを渡す先の SMTP サーバーの名前を決めるオプション。	04 10. メール
`Send-MailMessage -Port`	SMTP サーバーのポート番号を決めるオプション（25 など）。	04 10. メール
`Send-MailMessage -UseSsl`	STARTTLS で暗号化してから送るオプション。	04 10. メール
`Send-MailMessage -From`	送り主のメールアドレスを決めるオプション。	04 10. メール
`Send-MailMessage -To`	宛先のメールアドレスを決めるオプション。	04 10. メール
`Send-MailMessage -Subject`	件名を決めるオプション。	04 10. メール

コード	意味と使い方	初めて出てくる所
`Send-MailMessage -Body`	本文を決めるオプション。	04 10. メール
`Send-MailMessage -Encoding`	本文の文字コードを決めるオプション。日本語は `UTF8` にします（決めないと、文字化けすることがある）。	04 10. メール
`Get-Date -Format`	日時を、決めた形の文字列にするオプション。`Get-Date -Format "yyyyMMdd_HHmmss"` は「20261102_030000」の形（`yyyy` 年・`MM` 月・`dd` 日・`HH` 時・`mm` 分・`ss` 秒）。ファイルの名前に日時を入れる時に使います。	04 11. 毎晩のバックアップ
`Start-Transcript`	このあと画面に出る物を、全部ファイルに書き残し始めるコマンド。予定の仕事で動かすスクリプトの記録（何が起きたか）を残すのに使います。	04 11. 毎晩のバックアップ
`Start-Transcript -Path`	記録を書くファイルの場所を決めるオプション。	04 11. 毎晩のバックアップ
`Join-Path`	フォルダの場所とファイル名を、`\` でつないだ場所を作るコマンド。`Join-Path $PSScriptRoot "dev.env"` は「このスクリプトのフォルダの `dev.env`」。	04 11. 毎晩のバックアップ
`try`	`try { … } finally { … }` で、途中で失敗しても、`finally` の中（元のフォルダへ戻る など）を必ず動かす書き方。	04 11. 毎晩のバックアップ
`if`	PowerShell の条件の書き方。`if (-not (Test-Path $envFile)) { … }` は「ファイルが無ければ」。比べる時は `-eq`・`-ne`・`-not` などを使います。	04 11. 毎晩のバックアップ
`-ne`	PowerShell の「等しくない」の比べ方（not equal）。`$LASTEXITCODE -ne 0` は「失敗した（0 でない）か」。	04 11. 毎晩のバックアップ
`throw`	エラーを起こして、そこで止める命令。`throw "dev.env がありません"` のように、説明の文を付けます。`try … catch` の中なら `catch` へ、無ければスクリプトがそこで止まります。	04 11. 毎晩のバックアップ
`-ge`	PowerShell の「以上」の比べ方（greater or equal）。`$code.Count -ge 3` は「3 つ以上あるか」。	04 11. 毎晩のバックアップ
`Get-ChildItem -Filter`	`-Filter *.exe` で、名前が合う物（ここでは `.exe` で終わる物）だけを一覧にします。`*` は「何文字でもよい」という意味です。	04 11. 毎晩のバックアップ
`-lt`	PowerShell の「より小さい」の比べ方（less than）。`$_.LastWriteTime -lt $limit` は「決めた日時より古いか」。	04 11. 毎晩のバックアップ

コード	意味と使い方	初めて出てくる所
`Write-Output`	値を「出力」として次へ流す（最後なら画面に出す）コマンド。`Write-Output "こんにちは"`。文字列を 1 行に書くだけでも同じ働きをしますが、はっきり書く時に使います。	04 11. 毎晩のバックアップ
`Stop-Transcript`	`Start-Transcript` で始めた記録を終えるコマンド。	04 11. 毎晩のバックアップ
`exit`	スクリプトを、決めた終了コードで終えるコマンド。`exit $code` は、manage.py の終了コードをそのまま返します。	04 11. 毎晩のバックアップ
`catch`	`try { … } catch { … }` の `catch` の所は、`try` の中でエラーが起きた時だけ動きます。起きたエラーは `$_` で受け取れます（`$_.Exception.Message` で説明の文）。	04 11. 毎晩のバックアップ
`$ErrorActionPreference`	`$ErrorActionPreference = "Stop"` は、「エラーが起きたら、そこでスクリプトを止める」という設定です。付けないと、エラーが起きても次の行へ進んでしまいます（`.exe` のコマンドの失敗は止まらないので、`$LASTEXITCODE` も調べます）。	04 11. 毎晩のバックアップ
`$LASTEXITCODE`	`$LASTEXITCODE` は、最後に動かしたプログラム（`cargo`・`git`・`psql` など、`.exe` のコマンド）の「終了コード」が入る、特別な変数です。`0` なら成功、`0` 以外なら失敗です。`if ($LASTEXITCODE -ne 0) { … }` で、失敗した時の処理を書けます。	04 11. 毎晩のバックアップ
`.LastWriteTime`	`.LastWriteTime` は、ファイルを最後に書きかえた日時です。`Where-Object { $_.LastWriteTime -lt $limit }` で、古いファイルだけを選べます。	04 11. 毎晩のバックアップ
`New-ScheduledTaskAction`	予定の仕事で「何を動かすか」を作るコマンド。	04 11. 毎晩のバックアップ
`New-ScheduledTaskAction -Execute`	動かすプログラムを決めるオプション。`powershell.exe` は、PowerShell のスクリプトを動かす時。	04 11. 毎晩のバックアップ
`New-ScheduledTaskAction -Argument`	動かすプログラムに渡す引数を決めるオプション。`-NoProfile -ExecutionPolicy Bypass -File "D:\…\backup.ps1"` のように、動かすスクリプトを渡します。	04 11. 毎晩のバックアップ
`New-ScheduledTaskTrigger`	予定の仕事で「いつ動かすか」を作るコマンド。	04 11. 毎晩のバックアップ
`New-ScheduledTaskTrigger -Daily`	毎日動かすオプション。	04 11. 毎晩のバックアップ

コード	意味と使い方	初めて出てくる所
`New-ScheduledTaskTrigger -At`	動かす時刻を決めるオプション。`-At "03:00"` は夜中の 3 時（使う人がいない時間）。	04 11. 毎晩のバックアップ
`New-ScheduledTaskPrincipal`	予定の仕事を「だれとして動かすか」を作るコマンド。	04 11. 毎晩のバックアップ
`New-ScheduledTaskPrincipal -UserId`	動かす利用者を決めるオプション。`"SYSTEM"` は、Windows そのものの利用者（だれもログインしていなくても動く）。	04 11. 毎晩のバックアップ
`New-ScheduledTaskPrincipal -LogonType`	ログインのしかたを決めるオプション。`ServiceAccount` は、サービス用の利用者として動かす（パスワードを保存しなくてよい）。	04 11. 毎晩のバックアップ
`New-ScheduledTaskPrincipal -RunLevel`	権限の強さを決めるオプション。`Highest` は、管理者の権限で動かす。	04 11. 毎晩のバックアップ
`New-ScheduledTaskSettingsSet`	予定の仕事の細かい決まり（時間の上限など）を作るコマンド。	04 11. 毎晩のバックアップ
`New-ScheduledTaskSettingsSet -ExecutionTimeLimit`	仕事が動いてよい時間の上限を決めるオプション。過ぎたら止めて、止まったままの仕事が残らないようにします。	04 11. 毎晩のバックアップ
`New-TimeSpan`	時間の長さを作るコマンド。	04 11. 毎晩のバックアップ
`New-TimeSpan -Hours`	時間の長さを、時間（h）で決めるオプション。`New-TimeSpan -Hours 2` は 2 時間。	04 11. 毎晩のバックアップ
`New-ScheduledTaskSettingsSet -StartWhenAvailable`	決めた時刻にサーバーが止まっていた時、動けるようになったらすぐに動かすオプション。	04 11. 毎晩のバックアップ
`Register-ScheduledTask`	作った「何を・いつ・だれとして」を、タスクスケジューラーに登録するコマンド。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -TaskPath`	登録する先のフォルダを決めるオプション。`"\webapp\"` にまとめておくと、あとで探しやすくなります。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -TaskName`	予定の仕事の名前を決めるオプション。	04 11. 毎晩のバックアップ

コード	意味と使い方	初めて出てくる所
`Register-ScheduledTask -Action`	`New-ScheduledTaskAction` で作った「何を動かすか」を渡すオプション。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -Trigger`	`New-ScheduledTaskTrigger` で作った「いつ動かすか」を渡すオプション。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -Principal`	`New-ScheduledTaskPrincipal` で作った「だれとして動かすか」を渡すオプション。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -Settings`	`New-ScheduledTaskSettingsSet` で作った細かい決まりを渡すオプション。	04 11. 毎晩のバックアップ
`Register-ScheduledTask -Description`	予定の仕事の説明を書くオプション。あとで見た人が、何の仕事か分かるように書きます。	04 11. 毎晩のバックアップ
`Get-ScheduledTaskInfo -TaskPath`	調べる予定の仕事のフォルダを決めるオプション。	04 11. 毎晩のバックアップ
`Get-ScheduledTaskInfo -TaskName`	調べる予定の仕事の名前を決めるオプション。	04 11. 毎晩のバックアップ
`.FullName`	`Get-ChildItem` の結果の `.FullName` は、ファイルやフォルダの、`C:\` から始まる場所の全部です（`.Name` は名前だけ）。	04 11. 毎晩のバックアップ
`"</code> | "…" の中に " そのものを入れる時は、`` " ` と書きます（ ""` と 2 つ重ねても同じです）。`	04 12. 定期処理・ログ・監視	
`Rename-Item`	ファイルやフォルダの名前を変えるコマンド。`Rename-Item 今の名前 新しい名前`。	04 13. 動作確認・本番の開始・更新・切り戻し
`$( … )`	`"…$($t.Port)…"` の `$( )` は、「中の式やコマンドの結果を、文字の中に入れる」書き方です。`"$x.Name"` のように書くと `$x` だけが置きかわってしまうので、`.` の付いた物や計算は `$( )` で囲みます。	04 13. 動作確認・本番の開始・更新・切り戻し
`cd`	「change directory（フォルダを移る）」の略。`cd rust-study` で、今いるフォルダの中の `rust-study` へ移ります。`cd ..` は 1 つ上のフォルダへ、`cd ~` はホームフォルダ（`C:\Users\you`）へ戻ります。今いる場所は、行の頭の `PS C:\Users\you\rust-study>` の部分に表示されます。	05 7. ビルドして、まとめる

コード	意味と使い方	初めて出てくる所
`.\manage.ps1`	今いるフォルダの `manage.ps1`（この資料で作る、Django の管理コマンドを、本番の設定で動かすための小さなスクリプト）を動かす書き方。`.\manage.ps1 check --deploy` は、本番に出す前の設定の確かめです。	05 8. HTTP で動かすための設定
`param`	スクリプトが受け取る引数を決める書き方。`param([string[]] $ManageArgs)` は、スクリプトに渡した値を `$ManageArgs` に入れます。	05 9. サーバーに置き、Windows サービスにする
`Get-ChildItem -Path`	`Get-ChildItem` で、中身を見る場所を決めるオプション。`-Path $PSScriptRoot` は、このスクリプトがあるフォルダ。	05 9. サーバーに置き、Windows サービスにする
`Get-Content -LiteralPath`	`Get-Content` で、読むファイルの場所を、書いたとおりの文字で決めるオプション。`-Path` と違い、`[ ]` や `*` を特別な文字として扱わないので、どんな名前のファイルでも確実に読めます。	05 9. サーバーに置き、Windows サービスにする
`Push-Location`	今いるフォルダを覚えてから、指定したフォルダへ移るコマンド。あとで `Pop-Location` を打つと、覚えたフォルダへ戻れます。	05 9. サーバーに置き、Windows サービスにする
`Set-Item`	項目の値を設定するコマンド。`Set-Item -Path "Env:名前" -Value 値` で、この PowerShell の窓の中だけの環境変数を設定します。	05 9. サーバーに置き、Windows サービスにする
`Set-Item -Path`	`-Path "Env:名前"` で、どの項目（ここでは環境変数）を設定するかを指定します。	05 9. サーバーに置き、Windows サービスにする
`Set-Item -Value`	`-Value 値` で、設定する値を指定します。	05 9. サーバーに置き、Windows サービスにする
`finally`	`try { … } finally { … }` の `finally` の所は、`try` の中がうまくいっても、途中で失敗しても、必ず最後に動きます。元のフォルダへ戻る・記録を閉じる、などの後片づけに使います。	05 9. サーバーに置き、Windows サービスにする
`Remove-Item -Path`	`Remove-Item` で、消す物の場所を決めるオプション。`Remove-Item -Path "Env:NAME"` は、環境変数 `NAME` を消します（`Env:` は環境変数の場所）。	05 9. サーバーに置き、Windows サービスにする
`Remove-Item -ErrorAction`	`Remove-Item` で、エラーが起きた時の動きを決めるオプション。`-ErrorAction SilentlyContinue` は「無くてもエラーを出さずに次へ進む」。	05 9. サーバーに置き、Windows サービスにする

コード	意味と使い方	初めて出てくる所
`Pop-Location`	`Push-Location` で覚えたフォルダへ戻るコマンド。	05 9. サーバーに置き、Windows サービスにする
`$true`	`$true` は「はい（真）」を表す値です。`-Recurse:$true` のように、スイッチに値を渡す時にも使います。反対は `$false` です。	05 9. サーバーに置き、Windows サービスにする
`$PSScriptRoot`	`$PSScriptRoot` は、「今動いているスクリプト（ `.ps1` ）が置いてあるフォルダ」の場所が入った、特別な変数です。今いるフォルダがどこでも、スクリプトの隣のファイルを `Join-Path $PSScriptRoot "dev.env"` で見つけられます。	05 9. サーバーに置き、Windows サービスにする
`[Parameter()]`	`param( )` の中の `[Parameter(ValueFromRemainingArguments = $true)]` は、引数の受け取り方を決める印です。この指定なら、名前を付けずに渡した残りの引数を、全部その変数に受け取ります。`[Parameter(Mandatory)]` なら「必ず渡す」です。	05 9. サーバーに置き、Windows サービスにする
`[string[]]`	`[string[]]` は「文字列の並び（配列）」という型です。	05 9. サーバーに置き、Windows サービスにする
`[xml]`	`[xml](Get-Content x.xml)` は、XML のファイルの中身を読んで、`$svc.service.executable` のように、タグの名前でたどれる形にします。	05 9. サーバーに置き、Windows サービスにする
`@名前`	`コマンド @CurlArgs` のように、`$` の代わりに `@` を付けて変数を渡すと、変数に入れた並び（や表）の中身を、引数としてばらして渡せます（スプラッティング）。	05 9. サーバーに置き、Windows サービスにする
`param( … )`	スクリプトの最初に書く `param([string]$Base = "http://…")` は、「このスクリプトが受け取る引数」の決まりです。`.\smoke.ps1 -Base http://…` のように、名前を付けて渡せます。`=` の右は、渡さなかった時の値です。	05 9. サーバーに置き、Windows サービスにする
`Invoke-WebRequest -Headers`	送るリクエストに、見出し（ヘッダー）を足すオプション。`-Headers @{ Host = "webapp-stg.example.local" }` は、名前をDNS に登録する前に、IIS の振り分けを確かめる時に使います。	05 9. サーバーに置き、Windows サービスにする
`Test-Path`	ファイルやフォルダがあるかを調べるコマンド。あれば `True` 、無ければ `False` 。	05 13. 動作確認・更新・切り戻し

コード	意味と使い方	初めて出てくる所
`Remove-Item -Recurse`	フォルダを、中身ごと全部消すオプション。消した物は「ごみ箱」に入らず、元に戻せないので、場所をよく確かめてから使います。	05 13. 動作確認・更新・切り戻し
`Remove-Item -Force`	読み取り専用や隠しファイルも、確かめずに消すオプション。	05 13. 動作確認・更新・切り戻し
`function`	PowerShell で、自分の関数を作る書き方。`function New-LfFile($Path, $Lines) { … }` と書くと、`New-LfFile` をコマンドのように呼べます。	06 2. 切り分け: 手前から順に確かめる
`else`	`if (条件) { … } else { … }` の、条件に合わなかった時の所。`elseif (条件)` で、条件をつなげられます。	06 2. 切り分け: 手前から順に確かめる
`Get-Service -ErrorAction`	`Get-Service` で、エラーが起きた時の動きを決めるオプション。`-ErrorAction SilentlyContinue` は「そのサービスが無くても、赤いエラーを出さずに進む」。	06 2. 切り分け: 手前から順に確かめる
`-and`	「かつ」（両方とも `True` の時だけ `True`）。`$a -gt 0 -and $b -gt 0` のように、条件をつなぎます。	06 2. 切り分け: 手前から順に確かめる
`Get-NetTCPConnection -ErrorAction`	エラーが起きた時の動きを決めるオプション。待ち受けが無い時に出るエラーを、`-ErrorAction SilentlyContinue` で黙らせます。	06 2. 切り分け: 手前から順に確かめる
`Sort-Object -Unique`	並べかえたうえで、同じ物を 1 つにまとめるオプション。	06 2. 切り分け: 手前から順に確かめる
`Invoke-WebRequest -TimeoutSec`	答えを待つ時間の上限（秒）を決めるオプション。過ぎたらエラーにして、いつまでも止まらないようにします。	06 2. 切り分け: 手前から順に確かめる
`-gt`	PowerShell の「より大きい」の比べ方（greater than）。`$failed -gt 0` は「失敗が 1 つ以上あるか」。	06 2. 切り分け: 手前から順に確かめる
`[ValidateSet()]`	`[ValidateSet("prod", "stg")]` は、「この引数には、`prod` か `stg` だけを受け付ける」という印です。ほかの値を渡すと、動く前にエラーで止まります。	06 2. 切り分け: 手前から順に確かめる

コード	意味と使い方	初めて出てくる所
`[string]`	`[string]$Name` の `[string]` は、「この変数（引数）は文字列」という型の指定です。`[string]123` のように値の前に書くと、文字列に変えます。	06 2. 切り分け: 手前から順に確かめる
`[bool]`	`[bool]` は「はい・いいえ」（`$true`・`$false`）の型です。`[bool]$x` は、`$x` に何か入っていれば `$true`、空や `$null` なら `$false` にします。	06 2. 切り分け: 手前から順に確かめる
`.Service`	`$t.Service` は、上で作った表（`$targets`）から選んだ組の、`Service`（確かめるサービスの名前）の値です。	06 2. 切り分け: 手前から順に確かめる
`.Status`	`Get-Service` の結果の `.Status` は、サービスの状態（`Running` ＝動いている・`Stopped` ＝止まっている）です。	06 2. 切り分け: 手前から順に確かめる
`.Port`	`$t.Port` は、上で作った表（`$targets`）から選んだ組の、`Port`（アプリが待ち受けるポート番号）の値です。	06 2. 切り分け: 手前から順に確かめる
`.LocalAddress`	`.LocalAddress` は、待ち受けている、このパソコン側のアドレスです。`0.0.0.0` ならどこからでも、`127.0.0.1` ならこのパソコンの中からだけ受け付けます。	06 2. 切り分け: 手前から順に確かめる
`.Name`	`$t.Name` は、上で作った表（`$targets`）から選んだ組の、`Name`（サイトの名前）の値です。`$_.Name` のように、流れてきた物の名前を表すこともあります。	06 2. 切り分け: 手前から順に確かめる
`.Exception`	`catch` の中の `$_.Exception` は、起きたエラーの中身です。`.Message` で、エラーの説明文を取り出せます。	06 2. 切り分け: 手前から順に確かめる
`.Message`	`$_.Exception.Message` は、エラーの説明文です。	06 2. 切り分け: 手前から順に確かめる
`.DriveLetter`	`Get-Volume` の結果の `.DriveLetter` は、ドライブの文字（`C`・`D`）です。	06 2. 切り分け: 手前から順に確かめる
`.\check.ps1`	今いるフォルダの `check.ps1`（この資料で作る、サービス・待ち受け・応答・控えなどを、まとめて確かめるスクリプト）を動かす書き方。	06 2. 切り分け: 手前から順に確かめる
`-notmatch`	`-match` の逆で、正規表現に合わないかの比べ方。`$line -notmatch '='` は「`=` を含まない行か」。	06 3. 記録

コード	意味と使い方	初めて出てくる所
`-split`	文字列を、区切り（正規表現）で分けて並びにする演算子。`"a=b" -split '=', 2` は `"a"` と `"b"`（`2` は「多くても 2 つに分ける」）。	06 3. 記録
`$並び[0]`	`$list[0]` は、並びの 0 番目（最初の 1 つ）を取り出します。`$list[-1]` は最後の 1 つです。	06 3. 記録
`.StartMode`	`Win32_Service` の `.StartMode` は、サービスの起動のしかたです（`Auto` ＝パソコンの起動と一緒に・`Manual` ＝手で・`Disabled` ＝使わない）。	06 5. 症状別の詳しい説明
`.State`	`Win32_Service` の `.State` は、サービスの今の状態（`Running` など）です。	06 5. 症状別の詳しい説明
`Get-ChildItem -Directory`	`-Directory` で、フォルダだけを一覧にします（ファイルだけなら `-File`）。	06 5. 症状別の詳しい説明
`ForEach-Object`	`|` で流れてきた物を、1 つずつ処理するコマンド。`{ }` の中の `$_` が「今の 1 つ」を表します。`… | ForEach-Object { $_.Name }` は、1 つずつ名前を取り出します。	06 5. 症状別の詳しい説明
`Measure-Object`	受け取った物の数や、合計・平均などを計算するコマンド。	06 5. 症状別の詳しい説明
`Measure-Object -Sum`	決めた項目の合計を計算するオプション。`Measure-Object Length -Sum` は、ファイルの大きさの合計（`.Sum` で取り出す）。	06 5. 症状別の詳しい説明
`[pscustomobject]`	`[pscustomobject]@{ Folder = …; GB = … }` は、表（`@{ }`）から、列を持った 1 件の結果（オブジェクト）を作ります。並べて `Format-Table` や `Sort-Object` で扱えます。	06 5. 症状別の詳しい説明
`[Math]`	`[Math]` は、計算の関数が集まった .NET の型です。`[math]::Round(3.14159, 2)` で、小数点以下 2 けたに丸めます（3.14）。	06 5. 症状別の詳しい説明
`.Sum`	`Measure-Object Length -Sum` の結果の `.Sum` は、合計です（ここではファイルの大きさの合計）。	06 5. 症状別の詳しい説明
`mkdir`	「make directory（フォルダを作る）」の略。`mkdir ~\Documents\my-practice\01` のように、途中のフォルダもまとめて作れます（`~` はホームフォルダ `C:\Users\you`）。もう同じ名前のフォルダがあると、エラーで知らせます。	08 この資料について

コード	意味と使い方	初めて出てくる所
`Set-Content`	ファイルに文字を書くコマンド。`Set-Content memo.txt "1 行目"` で、ファイルを作るか、中身を置きかえます。後ろに書き足す時は `Add-Content` を使います。	08 この資料について
`Set-Content -Encoding`	`-Encoding utf8` で、ファイルを UTF-8 という文字コードで書きます。Windows PowerShell 5.1 は、指定しないと古い文字コード（Shift_JIS）で書くので、日本語を書く時は付けておくと安心です。	08 この資料について
`Set-Content -Value`	`-Value 中身` で、ファイルに書く中身を指定します。`@' … '@` で囲むと、何行もの文章をそのまま中身にできます（ヒア文字列）。	08 この資料について
`Get-Item`	ファイルやフォルダ 1 つの情報（名前・大きさ・日時）を取り出すコマンド。	08 この資料について
`$null`	`$null` は「何も無い」を表す値です。`if ($x -eq $null)` で「何も入っていないか」を調べられます。`$null = コマンド` や `コマンド | Out-Null` と書くと、コマンドの出力を捨てられます。	08 この資料について
`[datetime]`	`[datetime]` は「日時」の型です。`[datetime]::ParseExact("20261105 0200", "yyyyMMdd HHmm", $null)` のように、決まった形の文字から日時を作れます。	08 この資料について
`Read-Host`	キーボードから 1 行入力してもらうコマンド。`Read-Host "説明"` で、説明を出して待ちます。	参考 4. 証明書をサーバーに入れる
`Read-Host -AsSecureString`	入力した文字を画面に出さず、暗号化した形（`SecureString`）で受け取るオプション。パスワードを入れてもらう時に使い、コマンドの履歴にも残りません。	参考 4. 証明書をサーバーに入れる
`Import-PfxCertificate`	証明書と秘密鍵の入った `.pfx` のファイルを、Windows の証明書の置き場に入れるコマンド。	参考 4. 証明書をサーバーに入れる
`Import-PfxCertificate -FilePath`	入れる `.pfx` のファイルの場所を決めるオプション。	参考 4. 証明書をサーバーに入れる
`Import-PfxCertificate -CertStoreLocation`	入れる先の証明書の置き場を決めるオプション。`Cert:\LocalMachine\My` は、このパソコン全体の「個人」の置き場（IIS はここから使う）。	参考 4. 証明書をサーバーに入れる
`Import-PfxCertificate -Password`	`.pfx` のパスワード（`Read-Host -AsSecureString` で受け取った物）を渡すオプション。	参考 4. 証明書をサーバーに入れる
`.DnsNameList`	証明書の `.DnsNameList` は、その証明書が使える名前（ドメイン名）の一覧です。`.Unicode` で、名前を文字として取り出します。	参考 4. 証明書をサーバーに入れる

コード	意味と使い方	初めて出てくる所
`.Unicode`	`.DnsNameList` の 1 つずつの `.Unicode` は、名前を読める文字にした物です。	参考 4. 証明書をサーバーに入れる
`-contains`	左の並びの中に、右の値があるかの比べ方（`-in` と左右が逆）。`$names -contains "webapp.example.local"` は「その名前が含まれているか」。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding`	IIS のサイトに、受け取る住所・ポート・名前（バインド）を足すコマンド。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -Name`	バインドを足すサイトの名前を決めるオプション。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -BindingInformation`	足すバインドを決めるオプション。`"*:443:webapp.example.local"` は「どの住所でも・443 番（HTTPS）で・その名前の物」。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -Protocol`	通信の種類を決めるオプション。`https` にします。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -CertificateThumbPrint`	使う証明書を、その指紋（Thumbprint。証明書ごとに違う番号）で決めるオプション。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -CertStoreLocation`	証明書の置き場を決めるオプション。	参考 5. IIS に 443 番のバインドを足す
`New-IISSiteBinding -SslFlag`	HTTPS の細かい決まりを決めるオプション。`Sni` は「名前を見て、使う証明書を選ぶ」（同じ 443 番で、いくつもの名前を受け取れる）。	参考 5. IIS に 443 番のバインドを足す
`Get-IISSiteBinding`	IIS のサイトのバインド（受け取る住所・ポート・名前）の一覧を表示するコマンド。	参考 5. IIS に 443 番のバインドを足す
`Get-IISSiteBinding -Name`	調べるサイトの名前を決めるオプション。	参考 5. IIS に 443 番のバインドを足す
`.Thumbprint`	証明書の `.Thumbprint` は、証明書を見分けるための、長い 16 進数の文字（指紋）です。IIS に証明書を結び付ける時に使います。	参考 5. IIS に 443 番のバインドを足す

コード	意味と使い方	初めて出てくる所
`.NotAfter`	証明書の `.NotAfter` は、使える期限（この日時まで）です。	参考 11. 証明書の期限を切らさない

日時の書式

コード	意味と使い方	初めて出てくる所
`日時の書式`	`Get-Date -Format "yyyyMMdd_HHmm"` の `"…"` は、日時を文字にする時の形（書式）です。`yyyy`（年）・`MM`（月）・`dd`（日）・`HH`（時）・`mm`（分）・`ss`（秒）を並べ、ほかの文字（`_`・`-`）はそのまま入ります。2026 年 11 月 5 日 9 時 3 分なら `20261105_0903`。	04 11. 毎晩のバックアップ
`yyyy`	日時の書式の `yyyy` は、4 けたの年（`2026`）です。	04 11. 毎晩のバックアップ
`MM`	日時の書式の `MM`（大文字）は、2 けたの月（`01`〜`12`）です。小文字の `mm` は分なので、取り違えに注意します。	04 11. 毎晩のバックアップ
`dd`	日時の書式の `dd` は、2 けたの日（`01`〜`31`）です。	04 11. 毎晩のバックアップ
`HH`	日時の書式の `HH`（大文字）は、24 時間の形の時（`00`〜`23`）です。小文字の `hh` は 12 時間の形です。	04 11. 毎晩のバックアップ
`mm`	日時の書式の `mm`（小文字）は、2 けたの分（`00`〜`59`）です。	04 11. 毎晩のバックアップ

正規表現

コード	意味と使い方	初めて出てくる所
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	01 3. PowerShell の基本
`\s`	正規表現の `\s` は、「空白 1 つ」（スペース・タブ・改行など）を表します。`\s*` は「空白が 0 個以上」。	04 5. データベース

コード	意味と使い方	初めて出てくる所
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	04 5. データベース
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	04 5. データベース
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	05 6. アプリの作り
`\.`	正規表現の `\.` のように、記号の前に `\` を付けると、その記号を特別な意味にせず、文字そのものとして表します。`\.` は `.`、`\/` は `/` です。	05 6. アプリの作り
`[…]`	正規表現の `[ ]` は、「中に書いた文字の、どれか 1 つ」を表します。`[0-9a-zA-Z_-]` は「数字・英字・`_`・`-` のどれか」、`[.-]` は「`.` か `-`」。`-` で範囲を書けます。	05 6. アプリの作り
`{n,m}`	正規表現の `{6}` は「すぐ前の物が、ちょうど 6 回」、`{8,12}` は「8 回から 12 回」を表します。`\d{6}` は「数字 6 けた」。	05 6. アプリの作り
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	05 6. アプリの作り
`.`	正規表現の `.` は、「どんな 1 文字でも」を表します。`.` という文字そのものを表したい時は、`\.` と書きます。	05 6. アプリの作り
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	05 6. アプリの作り
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	05 6. アプリの作り
`(?=…)`	正規表現の `(?= )` は「この後ろに、こういう形が続く時だけ」、`(?! )` は「この後ろに、こういう形が続かない時だけ」を表します（先読み）。`^(?!api/|admin/).*$` は「`api/` や `admin/` で始まらない物、全部」。中の文字は、合わせた文字には入りません。	05 6. アプリの作り
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`|`	正規表現の `|` は、「左か右のどちらか」を表します。`api/|admin/` は「`api/` か `admin/`」。	05 6. アプリの作り
正規表現	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	05 6. アプリの作り
`(?:…)`	正規表現の `(?: )` は、取り出さないまとまりです。`(?:^|; )` は「行の頭か、`; ` のどちらか」をまとめるだけで、取り出す番号には数えません。	05 6. アプリの作り
`[^…]`	正規表現の `[^ ]` は、「中に書いた文字**以外**の 1 文字」を表します。`[^;]+` は「`;` 以外の文字が 1 つ以上」、`[^\x21-\x7E]` は「英数字と記号（ASCII の見える文字）以外」。	05 6. アプリの作り
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	05 6. アプリの作り
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	05 6. アプリの作り
`|`	正規表現の `|` は、「左か右のどちらか」を表します。`api/|admin/` は「`api/` か `admin/`」。	05 6. アプリの作り
`(…)`	正規表現の `( )` は、まとまり（グループ）を作ります。合った文字のうち、`( )` の中の部分だけを、あとから取り出せます（`m[1]`・`m.group(1)`・`$1` など）。`(…)?` のように、まとまりごとくり返しや省略もできます。	05 6. アプリの作り
`\d`	正規表現の `\d` は、「数字 1 つ（0〜9）」を表します。	06 3. 記録
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	06 3. 記録
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	06 3. 記録
`|`	正規表現の `|` は、「左か右のどちらか」を表します。`api/|admin/` は「`api/` か `admin/`」。	06 3. 記録

コマンド

コード	意味と使い方	初めて出てくる所
`hostname`	このパソコン（サーバー）の名前を表示するコマンド。	00 0. はじめの一歩: コンピューターの中身
`whoami`	今ログインしている利用者の名前（`ドメイン\名前` の形）を表示するコマンド。`whoami /groups` で、入っているグループも分かります。	00 0. はじめの一歩: コンピューターの中身
`quser`	このサーバーにログインしている人の一覧（名前・番号・状態・ログインした時刻）を表示するコマンド。	01 1. サーバーに入る
`logoff`	決めた番号の人を、ログオフさせるコマンド。`logoff 3` は、`quser` で見た番号 3 の人。保存していない作業は消えるので、必ず本人に確かめてから使います。	01 1. サーバーに入る
`icacls`	ファイルやフォルダの権限（だれが読める・書けるか）を表示する・変えるコマンド。`(R)` は読む、`(M)` は変える、`(F)` は全部できる、`(OI)(CI)` は中のファイルとフォルダにも引き継ぐ、という意味です。	01 5. ファイルとアクセス許可
`icacls /grant`	`icacls フォルダ /grant "名前:(OI)(CI)M"` は、利用者やグループに権限を与えます。`M` は変更（読む・書く・消す）、`RX` は読んで実行、`F` は全部。`(OI)(CI)` は、中のファイルとフォルダにも同じ権限を引き継ぐ印です。	01 5. ファイルとアクセス許可
`icacls /inheritance`	`/inheritance:r` は、上のフォルダから引き継いだ権限を外す指定です。`/inheritance:e` なら引き継ぎを戻します。権限を、自分で決めた物だけにする時に使います。	01 5. ファイルとアクセス許可
`icacls /remove`	`/remove 名前` は、その利用者やグループの権限を外します。	01 5. ファイルとアクセス許可
`sc.exe`	Windows のサービスの細かい設定をするコマンド。`sc.exe failure 名前 reset= 86400 actions= restart/60000/…` は「止まったら 60 秒後に始め直す。数えるのは 1 日ごとにやり直す」。PowerShell では `sc` だけだと別のコマンドになるので、必ず `.exe` まで書きます（`=` の後ろの空白も必要）。	01 6. サービス
`robocopy`	フォルダを丸ごとコピーする、Windows のコマンド（途中で切れても続きからやり直せる）。`/E` は中のフォルダも全部（空のフォルダも）、`/R:2` はしくじった時に 2 回までやり直す、`/W:5` はやり直す前に 5 秒待つ、`/NP` `/NFL` `/NDL` は、進み具合やファイルの名前を 1 つずつ表示しない、という意味です。終わった時の番号は、`0` 〜 `7` なら成功（`8` 以上が失敗）です。	01 12. ファイルを持ち込む

コード	意味と使い方	初めて出てくる所
`robocopy /R`	`/R:2` は、コピーに失敗した時に、やり直す回数です（決めないと 100 万回）。	01 12. ファイルを持ち込む
`robocopy /W`	`/W:5` は、やり直すまでに待つ秒数です（決めないと 30 秒）。	01 12. ファイルを持ち込む
`w32tm`	Windows の時刻合わせの状態を見る・変えるコマンド。`w32tm /query /status` で、どこと時刻を合わせているか（`Source`）と、最後に合わせた時刻が分かります。時刻がずれると、ログの読み合わせやログインの期限が狂います。	04 3. サーバーに入る・最初の確認
`w32tm /query`	`w32tm /query /status` は、パソコンの時計を、どこ（時刻のサーバー）に合わせているか、最後にいつ合わせたかを表示します。	04 3. サーバーに入る・最初の確認
`w32tm /status`	`/status` は、`w32tm /query` と一緒に使い、時刻合わせの今の状態を表示する指定です。	04 3. サーバーに入る・最初の確認
`notepad`	メモ帳を開くコマンド。`notepad D:\…\postgresql.conf` で、そのファイルをメモ帳で開きます。保存する時は、文字コードを変えないように注意します。	04 5. データベース
`D:\webapp\prod\service\webapp-prod-service.exe`	WinSW（ふつうの実行ファイルを Windows サービスにする道具）を、本番用の名前に変えて置いた物。`install` で、同じ名前の `.xml` の設定のとおりにサービスとして登録し、`uninstall` で取り外します。	04 7. アプリを Windows サービスにする
`D:\webapp\prod\app\webapp.exe`	アプリの実行ファイル（Rust で作った webapp）を、場所を書いて直接動かす書き方。`grant-role --employee-no 012345 --role it-admin` のように、サーバーとして動かす以外の、管理の仕事（社員に役割を付けるなど）を頼めます。	04 7. アプリを Windows サービスにする
`D:\webapp\prod\venv\Scripts\python.exe`	本番用の仮想環境（venv）の Python を、場所を書いて直接動かす書き方。`--version` で版を確かめます。どの Python で動いているかを取り違えないように、場所まで書きます。	05 4. Python を入れる
`robocopy /E`	`/E` は、中のフォルダも（空のフォルダも）全部コピーする指定です。	05 7. ビルドして、まとめる
`robocopy /XD`	`/XD .venv __pycache__` は、コピーしないフォルダの名前を並べる指定です。	05 7. ビルドして、まとめる
`robocopy /XF`	`/XF *.pyc` は、コピーしないファイルの名前（`*` で「何でも」）を並べる指定です。	05 7. ビルドして、まとめる

コード	意味と使い方	初めて出てくる所
`D:\webapp\stg\venv\Scripts\python.exe`	検証用の仮想環境の Python を、場所を書いて直接動かす書き方。`-m pip install …` は、その Python の pip で部品を入れる、`-c "…"` は、その場で 1 行の Python を動かす、という意味です。	05 9. サーバーに置き、Windows サービスにする
`D:\webapp\stg\service\webapp-stg-service.exe`	WinSW を、検証用の名前に変えて置いた物。`install` で、検証用のサービスとして登録します。	05 9. サーバーに置き、Windows サービスにする

設定ファイル（XML）

コード	意味と使い方	初めて出てくる所
`<!-- -->`	XML では、`<!--` から `-->` までがコメントです。	04 7. アプリを Windows サービスにする
`<service>`	WinSW（ふつうのプログラムを Windows のサービスにする道具）の設定で、いちばん外側の `<service>` の中に、サービスの設定を全部書きます。	04 7. アプリを Windows サービスにする
`<id>`	`<id>webapp-prod</id>` は、サービスの名前です。`Start-Service webapp-prod` のように、コマンドで使います。	04 7. アプリを Windows サービスにする
`<name>`	`<name>` は、サービスの一覧の画面に出る、人が読む名前です。	04 7. アプリを Windows サービスにする
`<description>`	`<description>` は、サービスの説明です。	04 7. アプリを Windows サービスにする
`<executable>`	`<executable>` は、サービスとして動かすプログラム（ `.exe` ）の場所です。	04 7. アプリを Windows サービスにする
`<workingdirectory>`	`<workingdirectory>` は、プログラムを動かす時の「今いるフォルダ」です。プログラムが、相対の場所でファイルを探すので、アプリのフォルダにします。	04 7. アプリを Windows サービスにする

コード	意味と使い方	初めて出てくる所
`<env>`	`<env name="PORT" value="8080" />` は、プログラムに渡す環境変数を 1 つ書くタグです。`/>` で終わるタグは、中身が無く、閉じるタグも要りません。	04 7. アプリを Windows サービスにする
`name=`	`<env name="…">` の `name` は、環境変数の名前です（IIS の設定では、決まりの名前）。	04 7. アプリを Windows サービスにする
`value=`	`<env … value="…">` の `value` は、環境変数の値です。	04 7. アプリを Windows サービスにする
`<startmode>`	`<startmode>Automatic</startmode>` は、パソコンの起動と一緒に、サービスも自動で動かす設定です。	04 7. アプリを Windows サービスにする
`<depend>`	`<depend>postgresql-x64-18</depend>` は、「このサービスより先に動かしておくサービス」です。データベースが動いてから、アプリを動かします。	04 7. アプリを Windows サービスにする
`<onfailure>`	`<onfailure action="restart" delay="10 sec" />` は、プログラムが落ちた時の動きです。1 回目は 10 秒後、2 回目は 30 秒後…のように、何回か書けます。	04 7. アプリを Windows サービスにする
`action=`	`action="restart"` は、落ちた時に、動かし直す指定です。IIS の `<action>` の `type` とは別の物です。	04 7. アプリを Windows サービスにする
`delay=`	`delay="10 sec"` は、動かし直すまでに待つ時間です。	04 7. アプリを Windows サービスにする
`<stoptimeout>`	`<stoptimeout>15 sec</stoptimeout>` は、サービスを止める時に、プログラムが自分で終わるのを待つ時間です。過ぎたら、強制的に止めます。	04 7. アプリを Windows サービスにする
`<logpath>`	`<logpath>` は、サービスのログ（プログラムが画面に出した文）を書くフォルダです。	04 7. アプリを Windows サービスにする
`<log>`	`<log mode="roll-by-size">` は、ログの残し方です。`roll-by-size` は、決めた大きさになったら、新しいファイルに切りかえる方式です。	04 7. アプリを Windows サービスにする

コード	意味と使い方	初めて出てくる所
`mode=`	`mode="roll-by-size"` は、ログを大きさで区切る指定です。	04 7. アプリを Windows サービスにする
`<sizeThreshold>`	`<sizeThreshold>10240</sizeThreshold>` は、1 つのログのファイルの大きさの上限（KB。10240 は 10MB）です。	04 7. アプリを Windows サービスにする
`<keepFiles>`	`<keepFiles>8</keepFiles>` は、残しておく古いログのファイルの数です。それより古い物は消えるので、ディスクがいっぱいになりません。	04 7. アプリを Windows サービスにする
`<?xml … ?>`	`<?xml version="1.0" encoding="UTF-8"?>` は、「これは XML のファイルで、文字の符号は UTF-8」という宣言です。ファイルのいちばん最初に書きます。	04 8. 受付
`<configuration>`	IIS の設定ファイル（`web.config`）の、いちばん外側のタグです。	04 8. 受付
`<system.webServer>`	`<system.webServer>` の中に、IIS（Windows の Web サーバー）の設定を書きます。	04 8. 受付
`<rewrite>`	`<rewrite>` の中に、来た要求の行き先を書きかえる決まり（URL Rewrite）を書きます。	04 8. 受付
`<rules>`	`<rules>` の中に、決まり（`<rule>`）を、上から順に並べます。	04 8. 受付
`<rule>`	`<rule name="…">` は、書きかえの決まり 1 つです。`<match>`（どの要求に）と `<action>`（どうするか）を中に書きます。	04 8. 受付
`stopProcessing=`	`stopProcessing="true"` は、「この決まりに合ったら、下の決まりは調べない」という指定です。	04 8. 受付
`<match>`	`<match url="(.*)" />` は、「どの URL に合わせるか」を正規表現で書くタグです。`(.*)` は「全部」で、`( )` の中に合った部分は、あとで `{R:1}` として使えます。	04 8. 受付
`url=`	`<match url="…">` の `url` は合わせる URL の形（正規表現）、`<action url="…">` の `url` は、取り次ぐ先や移す先の URL です。	04 8. 受付
`<action>`	`<action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />` は、合った要求をどうするかです。`Rewrite` は、利用者には見せずに、裏で別の場所（ここではアプリ）へ取り次ぎます。`Redirect` は、利用者のブラウザーに、別の URL へ行き直させます。	04 8. 受付

コード	意味と使い方	初めて出てくる所
`type=`	`<action type="Rewrite">` の `type` は、どう処理するか（`Rewrite` ＝裏で取り次ぐ・`Redirect` ＝行き直させる）です。	04 8. 受付
`<security>`	`<security>` の中に、IIS の守りの設定を書きます。	04 8. 受付
`<requestFiltering>`	`<requestFiltering>` は、受け付ける要求の決まり（大きさ・種類）の設定です。	04 8. 受付
`<requestLimits>`	`<requestLimits maxAllowedContentLength="104857600" />` は、1 回に受け付ける要求の大きさの上限です。	04 8. 受付
`maxAllowedContentLength=`	`maxAllowedContentLength` は、1 回に送れる中身の大きさの上限（バイト）です。`104857600` は 100MB。これより大きいファイルは、アプリに届く前に、IIS が断ります。	04 8. 受付
`{R:1}`	`{R:1}` は、`<match url="(.*)">` の `( )` の中に合った部分（1 番目）が入る場所です。`/notes/3` で来たら、`http://127.0.0.1:8080/notes/3` に取り次がれます。	04 8. 受付
`<arguments>`	`<arguments>` は、プログラムに渡す引数です（コマンドで打つ時の、プログラム名の後ろの部分）。	05 9. サーバーに置き、Windows サービスにする
`<conditions>`	`<conditions>` は、決まりを使うための、追加の条件（`<add>`）を並べる所です。	参考 9. 80 番で来たお願いを、443 番へ案内する
`<add>`	`<conditions>` の中の `<add input="{HTTPS}" pattern="off" />` は、「HTTPS ではない（`off`）時だけ」という条件です。	参考 9. 80 番で来たお願いを、443 番へ案内する
`input=`	`input="{HTTPS}"` は、調べる値です。`{HTTPS}` は、HTTPS で来たら `on`、そうでなければ `off` になる、IIS が用意した値です。	参考 9. 80 番で来たお願いを、443 番へ案内する
`pattern=`	`pattern="off"` は、`input` の値がこの形（正規表現）に合う時だけ、という条件です。	参考 9. 80 番で来たお願いを、443 番へ案内する

コード	意味と使い方	初めて出てくる所
`redirectType=`	`redirectType="Found"` は、行き直させる時の状態コードの種類です。`Found` は 302（一時的に移った）、`Permanent` は 301（ずっと移った）です。	参考 9. 80 番で来たお願いを、443 番へ案内する
`<serverVariables>`	`<serverVariables>` は、取り次ぐ時に、要求の見出しなどの値を書き足す所です。	参考 9. 80 番で来たお願いを、443 番へ案内する
`<set>`	`<set name="HTTP_X_FORWARDED_PROTO" value="https" />` は、取り次ぐ要求に、`X-Forwarded-Proto: https`（元は HTTPS で来た）という見出しを付けます。アプリは、これを見て、HTTPS で来たと分かります。	参考 9. 80 番で来たお願いを、443 番へ案内する

HTTP のやり取り

コード	意味と使い方	初めて出てくる所
`GET /… HTTP/1.1`	`GET /requests HTTP/1.1` は、ブラウザーがサーバーに送る要求（リクエスト）の 1 行目です。左から「何をしたいか（メソッド。GET は取ってきて）」「どこを（URL の場所）」「HTTP の版」です。	00 5. Web アプリの仕組み
`Host:`	`Host: webapp.example.local` は、「どの名前のサイトにアクセスしたいか」です。1 台のサーバーで、名前ごとに別のサイトを出す時に使われます。	00 5. Web アプリの仕組み
`User-Agent:`	`User-Agent:` は、「どのブラウザー（道具）から来た要求か」です。	00 5. Web アプリの仕組み
`Accept:`	`Accept: text/html` は、「こういう種類の答えがほしい」という希望です。	00 5. Web アプリの仕組み
`Cookie:`	`Cookie: session=AbC123...` は、ブラウザーが覚えていたクッキー（サイトから預かった札）を、サーバーに見せる見出しです。ログインの状態は、この札でサーバーに伝わります。	00 5. Web アプリの仕組み
`HTTP/1.1 200 OK`	`HTTP/1.1 200 OK` は、サーバーが返す答え（レスポンス）の 1 行目です。`200` は状態コードで「うまくいった」という意味です（404 は「無い」、500 は「サーバーの中で失敗」）。	00 5. Web アプリの仕組み

コード	意味と使い方	初めて出てくる所
`Content-Type:`	`Content-Type: text/html; charset=utf-8` は、「中身の種類と、文字の符号」です。ブラウザーは、これを見て HTML として表示します。	00 5. Web アプリの仕組み
`Content-Length:`	`Content-Length: 5120` は、中身の大きさ（バイト）です。	00 5. Web アプリの仕組み

psql

コード	意味と使い方	初めて出てくる所
`psql`	PostgreSQL に、コマンドでつないで SQL を打つ道具。`psql -h 127.0.0.1 -U postgres -d データベース名` でつなぎます（`-h` は相手の住所、`-U` は利用者の名前、`-d` はデータベース）。`\q` で抜けます。	02 2. ログインする
`psql -h`	`-h 127.0.0.1` は、つなぐ先のサーバーの場所（ホスト）です。`127.0.0.1` は「このパソコン自身」です。	02 2. ログインする
`psql -p`	`-p 5432` は、つなぐ先のポート番号です。PostgreSQL は、ふつう 5432 番で待ち受けます。	02 2. ログインする
`psql -U`	`-U postgres` は、どの利用者（ロール）としてつなぐかです。パスワードを聞かれたら、その利用者のパスワードを入れます。	02 2. ログインする
`psql -d`	`-d webapp_stg` は、どのデータベースにつなぐかです。	02 2. ログインする
`psql -c`	`-c "SELECT …;"` は、つないで、その SQL を 1 回だけ動かして、すぐに終わる指定です。確かめを 1 行で済ませる時に使います。	02 2. ログインする
`\c`	psql の `\c practice` は、つなぐデータベースを `practice` に切りかえる命令です。	02 3. 練習用のデータベースを作る
`\password`	psql の `\password 名前` は、その利用者（ロール）のパスワードを変える命令です。打つとパスワードを 2 回聞かれ、画面には表示されません（コマンドの履歴にも残りません）。	02 9. ユーザー
`pg_dump`	PostgreSQL のデータベースの中身を、まるごと 1 つのファイルに書き出す（控えを取る）コマンド。`-Fc` はあとで `pg_restore` で戻せる形、`-f` は書き出すファイルの場所。動かしている途中でも、ある瞬間の中身がそろった形で取れます。	02 11. バックアップとリストア
`pg_dump -h`	`-h 127.0.0.1` は、バックアップを取るサーバーの場所です。	02 11. バックアップとリストア

コード	意味と使い方	初めて出てくる所
`pg_dump -U`	`-U postgres` は、どの利用者としてつないで、バックアップを取るかです。	02 11. バックアップとリストア
`pg_dump -Fc`	`-Fc` は、「カスタム形式」（PostgreSQL 用に圧縮した形）でバックアップを書き出す指定です。この形は `pg_restore` で戻し、戻す時に表を選ぶこともできます。`-f ファイル` で書き出す先を決めます。	02 11. バックアップとリストア
`pg_restore`	`pg_dump -Fc` で取った控えを、データベースに戻すコマンド。`-d` で戻す先のデータベースを決めます。戻す練習は、本物とは別の、空のデータベースに対してします。	02 11. バックアップとリストア
`pg_restore -h`	`-h 127.0.0.1` は、戻す先のサーバーの場所です。	02 11. バックアップとリストア
`pg_restore -U`	`-U postgres` は、どの利用者としてつないで戻すかです。	02 11. バックアップとリストア
`pg_restore -d`	`-d practice_restore` は、戻す先のデータベースです（先に作っておきます）。	02 11. バックアップとリストア
`pg_restore --list`	`--list` は、戻さずに、バックアップの中身（表などの一覧）だけを表示します。バックアップが読めるかの確かめにも使えます。	02 11. バックアップとリストア
`psql --version`	`psql --version` は、psql（PostgreSQL につなぐ道具）の版を表示します。	04 5. データベース
`\l`	psql の `\l` は、データベースの一覧を表示する命令です。`\l webapp*` で、`webapp` で始まる物だけにしぼれます。	04 5. データベース
`\du`	psql の `\du` は、利用者（ロール）の一覧と、その権限を表示する命令です。	04 5. データベース
`\q`	psql の `\q` は、psql を終わる命令です。	04 5. データベース
`pg_restore --no-owner`	`--no-owner` は、表の持ち主を、バックアップを取った時の利用者に戻さず、戻している利用者にする指定です。別のサーバーや、別の名前の利用者で試しに戻す時に使います。	04 11. 毎晩のバックアップ
`pg_restore --no-privileges`	`--no-privileges` は、表の権限（`GRANT`）を戻さない指定です。	04 11. 毎晩のバックアップ
`pg_dump -d`	`-d webapp_prod` は、バックアップを取るデータベースです。	04 13. 動作確認・本番の開始・更新・切り戻し

コード	意味と使い方	初めて出てくる所
`pg_restore --clean`	`--clean` は、戻す前に、同じ名前の表などを消してから戻す指定です。今のデータが消えるので、戻す先をよく確かめます。	04 13. 動作確認・本番の開始・更新・切り戻し
`pg_restore --if-exists`	`--if-exists` は、`--clean` で消す時に、「あれば消す」（無くてもエラーにしない）にする指定です。	04 13. 動作確認・本番の開始・更新・切り戻し

SQL

コード	意味と使い方	初めて出てくる所
`SELECT`	表から行を取り出す SQL の命令。`SELECT 列, … FROM 表 WHERE 条件 ORDER BY 列` の形でよく使います。`SELECT *` は全部の列です。	00 7. データベースの基礎
`FROM`	`SELECT … FROM 表` の一部で、「どの表から」取り出すかを書きます。	00 7. データベースの基礎
`WHERE`	`SELECT … FROM 表 WHERE 条件` の、条件を書く所。`WHERE user_id = 1` は「持ち主が 1 番の行だけ」。`UPDATE`・`DELETE` で `WHERE` を忘れると、全部の行が変わる（消える）ので注意します。	00 7. データベースの基礎
`ORDER`	`ORDER BY 列` で、取り出した行を並べる書き方。`DESC` を付けると大きい順（新しい順）です。	00 7. データベースの基礎
`BY`	`ORDER BY`（並べる）・`GROUP BY`（まとめる）の一部で、「〜で」という意味です。	00 7. データベースの基礎
`DESC`	`ORDER BY 列 DESC` で、大きい順（新しい順）に並べます（小さい順は `ASC`）。	00 7. データベースの基礎
`LIMIT`	`LIMIT 件数` で、取り出す行の数の上限を決める書き方。`ORDER BY … LIMIT 20` は「並べて、先頭の 20 件だけ」。	00 7. データベースの基礎
`'文字'`	SQL では、文字の値を `'…'`（シングルクォート）で囲みます（`'2026-11-05'`・`'山田'`）。`"…"`（ダブルクォート）は表や列の名前に使う物なので、文字の値には使いません。中に `'` を入れる時は `''` と 2 つ書きます。	00 7. データベースの基礎
`--`	SQL では、`--` から行の終わりまでがコメントです。	00 7. データベースの基礎

コード	意味と使い方	初めて出てくる所
`current_database()`	PostgreSQL の `current_database()` は、今つないでいるデータベースの名前を返します。`current_user` は、今の利用者（ロール）の名前です。	02 2. ログインする
`now()`	PostgreSQL の `now()` は、今の日時を返す関数です。`DEFAULT now()` で、行を作った日時を自動で入れられます。	02 2. ログインする
`CREATE`	`CREATE ROLE`（ユーザーを作る）・`CREATE DATABASE`（データベースを作る）・`CREATE TABLE`（表を作る）のように、何かを新しく作る SQL の命令です。	02 3. 練習用のデータベースを作る
`DATABASE`	`CREATE DATABASE 名前` で、データベースを作ります。	02 3. 練習用のデータベースを作る
`ENCODING`	`ENCODING 'UTF8'` で、データベースの文字コードを UTF-8 にします（日本語を正しく入れるため）。	02 3. 練習用のデータベースを作る
`TEMPLATE`	`TEMPLATE template0` で、何も手を加えていない元の形から、データベースを作ります（文字コードを指定する時に必要）。	02 3. 練習用のデータベースを作る
`;`	SQL の `;` は、1 つの命令（文）の終わりの印です。psql では、`;` を打つまで命令が送られません（`;` を忘れると、続きを待つ `practice-#` の表示になります）。	02 3. 練習用のデータベースを作る
`TABLE`	SQL で「表」を表す言葉。`CREATE TABLE`（表を作る）・`DROP TABLE`（表を消す）のように使います。	02 3. 練習用のデータベースを作る
`PRIMARY`	`PRIMARY KEY`（主キー）の一部。主キーは、表の中で 1 行を必ず見分けられる列で、同じ値が 2 回入りません。ふつうは `id` 列にします。	02 3. 練習用のデータベースを作る
`KEY`	`PRIMARY KEY`（主キー）・`FOREIGN KEY`（よその表とつなぐ列）などの「キー」。	02 3. 練習用のデータベースを作る
`NOT`	SQL で「〜ではない」。`NOT NULL` は「この列は空（NULL）を許さない」という決まりです。	02 3. 練習用のデータベースを作る
`NULL`	SQL で「値が無い」ことを表す特別な値。0 や空の文字列とは違います。	02 3. 練習用のデータベースを作る
`UNIQUE`	「この列の値は、表の中で重ならない」という決まり。`email TEXT NOT NULL UNIQUE` は、同じメールアドレスを 2 回登録できなくします。	02 3. 練習用のデータベースを作る

コード	意味と使い方	初めて出てくる所
`REFERENCES`	「この列の値は、あの表の、あの列に必ずある値」という決まり（外部キー）。`user_id INTEGER REFERENCES users(id)` で、存在しないユーザーの番号を入れられなくなります。	02 3. 練習用のデータベースを作る
`DEFAULT`	列に値を入れなかった時に使う、決めておいた値（既定値）。`done INTEGER NOT NULL DEFAULT 0` は「指定が無ければ 0」。	02 3. 練習用のデータベースを作る
`GENERATED`	`GENERATED ALWAYS AS IDENTITY` は、行を足すたびに、番号を自動で振る列にする書き方です（PostgreSQL の新しい書き方。`ALWAYS` は「自分で番号を入れさせない」）。	02 3. 練習用のデータベースを作る
`ALWAYS`	`GENERATED ALWAYS AS IDENTITY` の一部で、「番号は必ずデータベースが振る（自分で入れようとするとエラー）」という意味です。	02 3. 練習用のデータベースを作る
`AS`	SQL で、列や表に別の名前を付ける書き方。`SELECT COUNT(*) AS notes` は、結果の列の名前を `notes` にします。`FROM users AS u` は、表を `u` と短く呼べるようにします。	02 3. 練習用のデータベースを作る
`IDENTITY`	`GENERATED ALWAYS AS IDENTITY` の一部で、「1, 2, 3… と自動で振る番号の列」という意味です。	02 3. 練習用のデータベースを作る
`CHECK`	「この列に入れてよい値の条件」を決める決まり。`CHECK (length(title) BETWEEN 1 AND 80)` は、題名を 1〜80 文字に限ります。合わない値を入れようとすると、データベースが断ります。	02 3. 練習用のデータベースを作る
`IN`	SQL で「このどれか」。`WHERE status IN ('queued', 'running')` は、状態がどちらかの行だけです。`CHECK (pinned IN (0, 1))` は、0 か 1 しか入れられない決まりです。	02 3. 練習用のデータベースを作る
`INSERT`	`INSERT INTO 表 (列, …) VALUES (値, …)` で、表に新しい行を足す SQL の命令。	02 3. 練習用のデータベースを作る
`INTO`	`INSERT INTO 表` の一部で、「この表へ」入れる、という意味。	02 3. 練習用のデータベースを作る
`VALUES`	`INSERT INTO 表 (列, …) VALUES (値, …)` の、入れる値を書く所。	02 3. 練習用のデータベースを作る
`SELECT *`	SQL の `SELECT *` の `*` は、「全部の列」という意味です。試しに中身を見る時は便利ですが、プログラムでは、使う列の名前を並べて書くほうが安全です（あとで列が増えても、読む物が変わりません）。	02 4. データを見る
`AND`	SQL で「かつ」。`WHERE a = 1 AND b = 2` は両方に合う行だけ。`BETWEEN 0 AND 100` の一部としても使います。	02 4. データを見る

コード	意味と使い方	初めて出てくる所
`DATE`	日付（年・月・日。時刻は無し）を入れる列の型。期限（`due_date`）などに使います。Rust では `time::Date` で受け取ります。	02 4. データを見る
`LIKE`	SQL のあいまい検索。`WHERE title LIKE '%買い物%'` は、`買い物` を含む行です（`%` は「何文字でも」、`_` は「どれか 1 文字」）。	02 4. データを見る
`%`	SQL の `LIKE '%買い物%'` の `%` は、「何文字でも、何でも合う」という印です（ワイルドカード）。`'買い物%'` なら「買い物で始まる」。`_` は「何でも 1 文字」です。`%` や `_` そのものを探したい時は、`ESCAPE` を使います。	02 4. データを見る
`IS`	SQL で NULL かどうかを調べる時の書き方。`WHERE x IS NULL`（無い）・`WHERE x IS NOT NULL`（ある）。NULL は `= NULL` では比べられません。	02 4. データを見る
`GROUP`	`GROUP BY 列` で、同じ値の行をまとめて、まとめごとに数える・足す時の書き方。`SELECT user_id, COUNT(*) FROM notes GROUP BY user_id` は、人ごとのノートの数です。	02 4. データを見る
`COUNT(*)`	SQL の `COUNT(*)` は、行の数を数えます。`GROUP BY status` と組み合わせると、`status` ごとの件数になります。	02 4. データを見る
`count()`	SQL の `count(*)` は、行の数を数える関数です（`COUNT(*)` と同じで、大文字・小文字はどちらでもかまいません）。	02 4. データを見る
`INDEX`	`CREATE INDEX 名前 ON 表 (列)` で、表に索引（本の索引のような、速く探すための仕組み）を作る SQL の命令。よく条件や並べかえに使う列に作ります。	02 4. データを見る
`ON`	SQL の「〜の時」「〜で」。`ON DELETE CASCADE` は「消された時は」、`JOIN … ON a.id = b.user_id` は「この条件で横につなぐ」、`CREATE INDEX … ON 表 (列)` は「この表の、この列に」という意味です。	02 4. データを見る
`EXPLAIN`	`EXPLAIN QUERY PLAN SELECT …` で、データベースがその SQL をどう実行するつもりか（索引を使うか、全部の行を見るか）を表示します。	02 4. データを見る
`JOIN`	2 つの表を、条件で横につないで取り出す書き方。`FROM notes JOIN users ON users.id = notes.user_id` は、ノートと、その持ち主を 1 行に並べます。	02 5. テーブルをつなぐ

コード	意味と使い方	初めて出てくる所
`表.列`	SQL の `n.title`・`users.email` のように `表.列` と書くと、「どの表の列か」をはっきりさせられます。いくつもの表をつなぐ（`JOIN`）時は、同じ名前の列（`id` など）があるので、こう書きます。`notes AS n` のように、表に短い別名を付けて使います。	02 5. テーブルをつなぐ
`LEFT`	`LEFT JOIN` は、左の表の行は、相手が無くても全部残す JOIN。「ノートが 0 件の人も一覧に出す」時に使います。	02 5. テーブルをつなぐ
`RETURNING`	`INSERT … RETURNING id` で、入れた（変えた）行の値（作られた番号など）を、その場で返してもらう書き方。	02 6. データを入れる・直す・消す
`UPDATE`	`UPDATE 表 SET 列 = 値 WHERE 条件` で、行の中身を書きかえる SQL の命令。`WHERE` を忘れると、全部の行が変わります。	02 6. データを入れる・直す・消す
`SET`	`UPDATE` の中で、「どの列を、どの値にするか」を書く所。	02 6. データを入れる・直す・消す
`DELETE`	`DELETE FROM 表 WHERE 条件` で、表の行を消す SQL の命令。`WHERE` を忘れると、全部の行が消えます。`ON DELETE CASCADE` の中では「消された時」という意味です。	02 6. データを入れる・直す・消す
`BEGIN`	トランザクションを始める SQL の命令。ここから `COMMIT`（確定）か `ROLLBACK`（取り消し）までが、1 つのまとまりになります。	02 7. トランザクション
`ROLLBACK`	トランザクションの中でした変更を、全部取り消す SQL の命令。	02 7. トランザクション
`ROLE`	PostgreSQL のユーザー（ログインする名前と、持つ権限のまとまり）。`CREATE ROLE webapp_dev_app LOGIN CREATEDB;` は「ログインでき、データベースを作れるユーザー」を作ります。	02 9. ユーザー
`LOGIN`	`CREATE ROLE … LOGIN` で、そのユーザーがログインできるようにします。	02 9. ユーザー
`GRANT`	利用者（ロール）に、権限を与える命令。`GRANT CONNECT ON DATABASE webapp_prod TO webapp_readonly;` は「webapp_readonly に、webapp_prod へつなぐことを許す」。	02 9. ユーザー
`CONNECT`	`GRANT CONNECT ON DATABASE …` の `CONNECT` は、「そのデータベースにつなぐ」権限です。	02 9. ユーザー
`TO`	`RENAME TO 新しい名前` の「〜へ」の部分です。	02 9. ユーザー
`SCHEMA`	スキーマは、データベースの中の、表をまとめる入れ物（ふだんは `public`）。`GRANT USAGE ON SCHEMA public TO …` は「public の中の物を使ってよい」。	02 9. ユーザー

コード	意味と使い方	初めて出てくる所
`ALL`	`UNION ALL` は、2 つの結果を、重なりも消さずにそのままつなげる書き方です。	02 9. ユーザー
`TABLES`	`ON ALL TABLES IN SCHEMA public` は「public の中の、今ある表すべて」。`GRANT SELECT ON ALL TABLES …` で、全部の表を読めるようにします。	02 9. ユーザー
`ALTER`	`ALTER TABLE 表 …` で、もうある表の形を変える SQL の命令（列を足す・名前を変える など）。	02 9. ユーザー
`PRIVILEGES`	`ALTER DEFAULT PRIVILEGES … GRANT SELECT ON TABLES TO …` は、「これから作られる表にも、自動で同じ権限を付ける」という決まりを作る命令です。これが無いと、あとで足した表は読めません。	02 9. ユーザー
`FOR`	`SELECT … FOR UPDATE` は、読んだ行に鍵をかける書き方です。同じ行を、別の人が同時に変えようとしても、こちらのトランザクションが終わるまで待たせるので、「2 人が同時に承認して 2 回進む」ことを防げます。	02 9. ユーザー
`SHOW`	PostgreSQL の設定の、今の値を表示する命令。`SHOW listen_addresses;` は、どの住所で待ち受けているか。	02 10. 設定ファイル
`SQL`	ここでは SQL の命令ではなく、`AS SQL` で列に付けた名前（見出し）です。`left(query, 60) AS SQL` は「動いている SQL の先頭 60 文字を、SQL という見出しで出す」。	02 12. 運用で使う SQL
`pg_size_pretty()`	PostgreSQL の `pg_size_pretty(数)` は、バイトの数を、`12 MB` のような読みやすい形にします。	02 12. 運用で使う SQL
`pg_database_size()`	PostgreSQL の `pg_database_size(名前)` は、データベースの大きさ（バイト）を返します。	02 12. 運用で使う SQL
`pg_total_relation_size()`	PostgreSQL の `pg_total_relation_size(表)` は、表の大きさ（索引なども合わせたバイト）を返します。	02 12. 運用で使う SQL
`pg_terminate_backend()`	PostgreSQL の `pg_terminate_backend(番号)` は、その番号（PID）のつながりを、強制的に切ります。止まらない問い合わせを止める時に使います。	02 12. 運用で使う SQL
`DROP`	`DROP TABLE 表`・`DROP INDEX 索引` で、表や索引を消す SQL の命令。中のデータも消えるので注意します。	02 13. 練習の片付け
`OWNER`	`OWNER ユーザー` で、そのデータベースの持ち主を決めます。持ち主は、中の表を自由に作れます。	04 5. データベース

コード	意味と使い方	初めて出てくる所
`REVOKE`	利用者（ロール）から、権限を取り上げる命令（`GRANT` の逆）。`REVOKE ALL ON DATABASE webapp_prod FROM PUBLIC;` は「だれでもつなげる、の決まりを取り上げる」。	04 5. データベース
`PUBLIC`	PostgreSQL で「すべての利用者」を表す名前。初めの状態では、だれでもデータベースにつなげるので、`REVOKE … FROM PUBLIC` で閉じます。	04 5. データベース

Python の道具

コード	意味と使い方	初めて出てくる所
`.\.venv\Scripts\python.exe`	仮想環境（`.venv`）の中の Python を、場所を書いて動かす書き方。`.\.venv\Scripts\python.exe ファイル.py` でプログラムを動かし、`.\.venv\Scripts\python.exe -m pip install -r requirements.txt` で、その仮想環境に部品を入れます。仮想環境を「入る」操作（Activate）をしなくても、確実にその Python が使われます。	05 7. ビルドして、まとめる

Node.js

コード	意味と使い方	初めて出てくる所
`npm ci`	`package-lock.json` に書かれたとおりの版で、部品を入れ直すコマンド（`node_modules` を一度消してから入れる）。`npm install` と違い、`package-lock.json` を書きかえないので、いつ・どこで入れても同じになります。	05 7. ビルドして、まとめる
`npm run`	`package.json` の `scripts` に書いたコマンドを動かすコマンド。`npm run dev` は開発用のサーバーを起動し、`npm run build` は本番用のファイルを作ります。止める時は `Ctrl` + `C` です。	05 7. ビルドして、まとめる

cargo のコマンド

コード	意味と使い方	初めて出てくる所
`cargo build`	プログラムを翻訳（コンパイル）して実行ファイルを作るだけで、動かしはしません。でき上がった物は `target\debug\` に入ります。`cargo build --release` だと、速く動く本番用の物が `target\release\` にできます。	04 6. アプリを作る
`cargo build --release`	`cargo build --release` は、時間をかけて最適化した、速い本番用の実行ファイルを作ります（動かしはしません）。できた物は `target\release` に置かれます。	04 6. アプリを作る
`cargo build --locked`	`--locked` は、`Cargo.lock` に書いてある部品の版のまま作り、`Cargo.lock` を書きかえない指定です。版がずれていたらエラーで止まるので、いつ作っても同じ物ができます。本番用やテストの時に付けます。	04 6. アプリを作る

設定ファイル（TOML）

コード	意味と使い方	初めて出てくる所
`#`	TOML（`Cargo.toml` などの設定ファイルの書き方）では、`#` から行の終わりまでがコメントです。	04 6. アプリを作る
`[target.…]`	`.cargo/config.toml` の `[target.x86_64-pc-windows-msvc]` は、「その種類のパソコン向けに作る時だけの設定」の見出しです。	04 6. アプリを作る
`rustflags =`	`rustflags = ["-C", "target-feature=+crt-static"]` は、Rust のコンパイラーに渡す追加の指定です。`+crt-static` は、Windows の実行ファイルに、C の部品（ランタイム）を中に入れる指定で、Visual C++ の部品が入っていないサーバーでも動くようになります。	04 6. アプリを作る
`[ … ]`	`["rustfmt", "clippy"]` のように `[ ]` の中に `,` で区切って並べると、いくつもの値の並び（配列）になります。	04 6. アプリを作る

コメント

コード	意味と使い方	初めて出てくる所
`//!`	`//!` で始まるコメントは、そのファイル（モジュール）全体の説明書です。ファイルのいちばん上に書きます。`///`（すぐ下の関数や型の説明）と使い分けます。	04 6. アプリを作る
`///`	`///` で始まるコメントは、すぐ下の関数や型の「説明書」（ドキュメントコメント）です。`cargo doc` で説明のページになり、VS Code では、名前にマウスを乗せると表示されます。ふつうのコメント（`//`）とちがい、何の説明かが決まっています。	04 6. アプリを作る
`//`	`//` から行の終わりまでは「コメント」（人が読むためのメモ）で、プログラムとしては無視されます。`// OS の種類を表示する` のように、何をしているかを日本語で書いておくと、あとで読み返す時に助かります。	04 6. アプリを作る

変数

コード	意味と使い方	初めて出てくる所
`const`	ずっと変わらない値（定数）に名前を付ける書き方。`const MAX_TITLE_CHARS: usize = 80;`。名前は大文字と `_` で書き、型（`usize` など）を必ず書きます。プログラムのどこからでも使えます。	04 6. アプリを作る
`let`	値に名前を付けて覚えておく（変数を作る）書き方。`let age = 20;` で、`age` という名前で 20 を覚えます。最初は「変えられない」変数です。あとで変えたい時は `let mut age = 20;` のように `mut` を付けます。	04 6. アプリを作る

文字列

コード	意味と使い方	初めて出てくる所
`str`	`&str` の `str` は「文字列の中身」を表す型で、ふつうは `&str`（借りた文字列）の形で使います。`"こんにちは"` のように `"` で囲んで直接書いた文字列は `&str` です。自分で持って、あとから書き足せる文字列は `String` です。	04 6. アプリを作る

Vec（並び）

コード	意味と使い方	初めて出てくる所
`.get()`	番号やキーで中身を取り出すメソッド。`v.get(10)` は、範囲の外なら止まらずに `None` を返します（`v[10]` はプログラムが止まる）。文字列の `s.get(0..3)` は、文字の区切りでない位置なら `None` です。`HashMap` の `map.get("キー")` も、無ければ `None` です。	04 6. アプリを作る
`.remove()`	`Vec` から i 番目を取り除いて、その値を返すメソッド。後ろの物が前に詰められます。`HashMap` の `.remove(キー)` は、そのキーの組を取り除き、値を `Option` で返します。	04 6. アプリを作る

条件

コード	意味と使い方	初めて出てくる所
`else`	`if` の条件が合わなかった時の処理を書く所。`else if 条件 { … }` で、条件をいくつも続けられます。	04 6. アプリを作る

くり返し

コード	意味と使い方	初めて出てくる所
`for`	並びや範囲から 1 つずつ取り出して、くり返す書き方。`for n in 1..=100 { … }` は、`n` に 1, 2, ..., 100 を順に入れて、中を 100 回動かします。`for x in &v` で、`Vec` の中身を借りて回せます。	04 6. アプリを作る

Option

コード	意味と使い方	初めて出てくる所
`Option`	「値があるかもしれないし、無いかもしれない」を表す型。中身は `Some(値)` か `None`。`Option<u32>` は「整数があるかもしれない」。取り出す時は `match`・`if let`・`unwrap_or` などを使います。	04 6. アプリを作る

コード	意味と使い方	初めて出てくる所
`Some`	`Option`（値が「ある」か「無い」か）の、「ある」の方。`Some(値)` の形で中身を包みます。`match` で `Some(name) => { … }` と書くと、中身を `name` という名前で取り出して使えます。「無い」の方は `None` です。	04 6. アプリを作る
`None`	`Option` の「値が無い」側。`Some(値)`（値がある）か `None`（無い）のどちらかになります。Rust には「null」が無い代わりに、無いかもしれない値は必ず `Option` で表し、`match` などで両方の場合を書きます。	04 6. アプリを作る
`let … else`	`let Some(x) = 値 else { return …; };` は、「`Some` の形なら中身を `x` に入れて先へ進み、そうでなければ `else` の中で抜ける」という書き方です。`else` の中は、必ず `return` や `continue` などで抜けます。思ったとおりでない時に早めに抜けて、残りの処理を平らに書けます。	04 6. アプリを作る

エラー処理

コード	意味と使い方	初めて出てくる所
`.ok()`	`Result` を `Option` に変えるメソッド。`Ok(値)` は `Some(値)` に、`Err(…)` は `None` になります（エラーの中身は捨てる）。`"42".parse().ok()` のように、「失敗したら無かったことにする」時に使います。	04 6. アプリを作る
`Ok`	`Result` の「うまくいった」の方。`Ok(値)` の形で結果を包みます。`Ok(())` は「うまくいった。返す値は特に無い（`()` は空っぽの値）」という意味で、`Result<()>` を返す関数の最後によく書きます。	04 6. アプリを作る
`?`	`Result` や `Option` の後ろに付ける記号。「成功なら中身を取り出して先へ進む。失敗なら、その場で関数を終えて、エラーを呼び出し元へ返す」という意味です。`let text = fs::read_to_string("a.txt")?;` のように使います。`?` を使える関数は、戻り値が `Result`（または `Option`）の関数だけです。	04 6. アプリを作る

関数

コード	意味と使い方	初めて出てくる所
`fn`	関数（ひとまとまりの処理に名前を付けた物）を作る書き方。`fn main() { … }` の `main` は、プログラムを動かすと最初に動く特別な関数です。`fn add(a: i32, b: i32) -> i32 { a + b }` は「整数を 2 つ受け取って、整数を返す関数 `add`」。`{ }` の中が中身で、最後の `;` の無い式が戻り値になります。	04 6. アプリを作る
`move`	クロージャが使う外の変数を、借りるのではなく「持っていく」印。`thread::spawn(move || { … })` のように、別のスレッドに値を渡す時に必要です。	04 6. アプリを作る
`return`	関数を、その場で終えて値を返す命令。`return Err(…);` のように、途中で抜ける時に使います。Rust では、関数の最後の `;` の無い式が自動で戻り値になるので、最後の行では `return` を書かないのがふつうです。	04 6. アプリを作る
`->`	`fn main() -> std::io::Result<()>` の `->` は、「この関数が返す値の型」を書く印です。`fn add(a: i32, b: i32) -> i32` なら「`i32` を返す」。`->` が無い関数は、何も返しません（`()` という「空」を返します）。	04 6. アプリを作る

型とメソッド

コード	意味と使い方	初めて出てくる所
`struct`	いくつかの値をまとめて、新しい型を作る書き方（構造体）。`struct Note { id: u32, title: String }` で、「番号と題名を持つノート」という型ができます。`Note { id: 1, title: "買い物".to_string() }` で 1 つ作ります。	04 6. アプリを作る
`impl`	2 つの使い方があります。① `impl 型 { … }` で、その型にメソッド（関数）を付ける。② `impl View` のように型の場所に書くと、「`View` の機能を持つ何かの型」という意味になります（戻り値の型を細かく書かずに済む）。	04 6. アプリを作る
`self`	メソッドの中で「このもの自身」を表す名前。`impl` の中の `fn label(&self)` の `&self` は「自分を借りて読むだけ」、`&mut self` は「自分を書きかえる」という意味です。`use` の中の `self`（例: `use std::io::{self, Write};`）は「`io` そのもの」を表します。	04 6. アプリを作る

コード	意味と使い方	初めて出てくる所
`impl トレイト`	`-> Result<impl View>` の `impl View` は、「`View` という決まり（トレイト）を満たす、何かの型」という意味です。本当の型の名前は長くて書けないので、「`View` として使える物を返す」とだけ書きます。引数の `x: impl Display` なら、「表示できる物なら何でも受け取る」です。	04 6. アプリを作る
`<T>`	`fn longest<'a>(…)` や `struct Wrapper<T>` の、名前のすぐ後ろの `< >` は、「型（やライフタイム）を、使う時に決める」ための入れ物（ジェネリクス）です。`T` は「何かの型」の仮の名前で、使う時に `i32` や `String` などに決まります。	04 6. アプリを作る
`.build()`	設定を少しずつ足していく「組み立て係」（ビルダー）の最後に呼んで、でき上がった物を作るメソッド。`Router::builder()…build()` の形で、ルーターなどを作る時に使います。	04 6. アプリを作る

モジュール

コード	意味と使い方	初めて出てくる所
`use`	ほかの場所にある道具に、短い名前で呼べるようにする宣言。`use std::env;` と書くと、`std::env::args()` を `env::args()` と短く書けます。`std` は Rust に最初から付いている道具箱（標準ライブラリ）です。	04 6. アプリを作る
`pub`	「外から使ってよい」（公開）という印。`pub fn title_length(…)` は、ほかのファイル（モジュール）からも呼べる関数になります。付けないと、同じモジュールの中からしか使えません。	04 6. アプリを作る
`::`	`std::env` の `::` は、「〜の中の」を表す区切りです。`std::env::args()` は「標準ライブラリ（`std`）の中の `env` の中の `args` 関数」という意味です。型の関数（`String::new()`）や、enum の種類（`Ordering::Less`）を指す時にも使います。	04 6. アプリを作る
`crate`	「このプロジェクト（クレート）のいちばん上」を表す言葉。`use crate::note::Note;` は「このプロジェクトの `note` モジュールの `Note` を使う」という意味です。	04 6. アプリを作る

所有権と借用

コード	意味と使い方	初めて出てくる所
`&`	値の前の `&` は「貸す（借用）」の印です。`fs::write(&relative, …)` は、`relative` を手放さずに、見せるだけ貸します。型に付けた `&str`・`&Vec<i32>` は「借りた値」の型です。	04 6. アプリを作る
`'a`	`&'a str` の `'a` は、「借りている期間」（ライフタイム）に付けた名前です。`fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` は、「返す参照は、`x` と `y` の両方が有効な間だけ使える」という約束を表します。	04 6. アプリを作る

スマートポインタ

コード	意味と使い方	初めて出てくる所
`Box`	値をヒープ（大きな物を置く広い場所）に置き、その場所を指す「持ち手」を手元に置く型。`Box<dyn Notifier>` のように、違う型の物を同じ並びに入れる時によく使います。	04 6. アプリを作る

時間

コード	意味と使い方	初めて出てくる所
`time::Duration::try_from()`	標準ライブラリの `Duration` を、`time` の `Duration` に変える関数。大きすぎて入らない時は `Err` を返すので、`?` で受け取ります。	04 6. アプリを作る
`Duration`	時間の長さを表す型。`Duration::from_secs(5)` で 5 秒。`{:?}` で表示すると `5s` や `200ms` のように単位つきで出ます。	04 6. アプリを作る
`std::time::Duration::from_secs()`	秒数ぶんの長さ（`Duration`）を作る関数。`Duration::from_secs(5)` は 5 秒です。	04 6. アプリを作る

非同期

コード	意味と使い方	初めて出てくる所
`async`	「待つ所で、ほかの仕事に順番をゆずれる関数」にする印。`async fn root(…)` のように付けます。呼ぶ側は `.await` で結果を待ちます。Web サーバーが、たくさんの人を同時にさばけるのは、この仕組みのおかげです。	04 6. アプリを作る
`Box::pin()`	値を、メモリーの決まった場所に固定して箱に入れる関数。`Box::pin(async move { … })` は、「あとで待つ仕事（Future）」を、トレイトの決まりどおりの形で返すための書き方です。	04 6. アプリを作る

クッキー

コード	意味と使い方	初めて出てくる所
`.override_same_site()`	送るクッキーの `SameSite`（ほかのサイトから来たリクエストにも付けて送るか）を決めるメソッド。`SameSite::Lax` は「ほかのサイトからのリンクで開いた時は送るが、ほかのサイトのフォームからの POST には付けない」で、CSRF を防ぐ助けになります。	04 6. アプリを作る
`.override_http_only()`	`HttpOnly`（JavaScript から読めないようにする印）を付けるかを決めるメソッド。`true` にすると、XSS でクッキーを盗まれにくくなります。	04 6. アプリを作る
`.override_secure()`	`Secure`（HTTPS の時だけ送る印）を付けるかを決めるメソッド。この資料のサーバーは HTTP なので `false` にしています（付けると、ブラウザーがクッキーを送らず、ログインが続かない）。	04 6. アプリを作る
`.override_path()`	クッキーを送る URL の範囲を決めるメソッド。`"/"` は、そのサイトの全部の URL。	04 6. アプリを作る
`.value_trimmed()`	クッキーの値を、前後の `"` を取り除いて返すメソッド。	04 6. アプリを作る
`.override_max_age()`	クッキーが使える長さを決めるメソッド。セッションの長さと同じにします。	04 6. アプリを作る
`.add()`	クッキーの入れ物（`CookieJar`。この答えで送るクッキーの束）に、新しいクッキーを足すメソッド。`.add(Cookie::new(名前, 値))` で足した物が、答えの `Set-Cookie` の見出しになってブラウザーに届きます。	04 6. アプリを作る

コード	意味と使い方	初めて出てくる所
`Cookie::new()`	名前と値を決めて、送るクッキーを作る関数。`Cookie::new(NAME, token.encode())`。	04 6. アプリを作る
`Cookie`	クッキー 1 つ（名前と値と、使う範囲・期限などの設定）を表す型。	04 6. アプリを作る
`Cookies`	今のリクエストのクッキーを読み書きする入れ物（クッキーの瓶）。`jar.add(…)` で足し、`jar.get("名前")` で読みます。	04 6. アプリを作る
`SameSite`	クッキーの決まりの 1 つ。ほかのサイトから来た頼みに、このクッキーを付けるかを決めます。`Lax` は、ほかのサイトからのリンクでの移動には付けるが、フォームの送信などには付けない設定です。	04 6. アプリを作る
`Lax`	`SameSite = Lax` は、ほかのサイトからのリンクで来た時（ふつうの移動）にはクッキーを送り、ほかのサイトからのフォームの送信には送らない設定です。	04 6. アプリを作る

認証

コード	意味と使い方	初めて出てくる所
`.encode()`	`base64` の部品で、バイトの並びを、URL に入れても安全な文字（Base64）に変えるメソッド。`URL_SAFE_NO_PAD.encode(バイト)`。	04 6. アプリを作る

Topcoat

コード	意味と使い方	初めて出てくる所
`topcoat`	Topcoat の道具のコマンド（`cargo install topcoat-cli` で入る）。`topcoat dev`（コードを直すと自動で作り直して読み直す開発用の起動）・`topcoat asset bundle`（CSS などの束を作る）などがあります。	04 6. アプリを作る
`topcoat asset bundle --release`	`topcoat asset bundle --release` は、CSS などのアセットの束を、本番用（`target\release\assets`）に作ります。`cargo build --release` と組にして使います。	04 6. アプリを作る
`Token::decode()`	クッキーに入っていた文字列から、セッションの札（`Token`）に戻す関数。形がおかしければ `Err` です。	04 6. アプリを作る

コード	意味と使い方	初めて出てくる所
`Cx`	Topcoat の「今のリクエストの情報（URL・クッキー・セッションなど）が入った箱」。ページの関数の引数 `cx: &Cx` で受け取り、`href!(…).resolve(cx)` などで使います。	04 6. アプリを作る
`Token`	`topcoat::session::Token` は、セッションの印（でたらめな長い文字列）の型。ブラウザーのクッキーに入れ、サーバーには、そのハッシュだけを保存します。	04 6. アプリを作る
`TokenStore`	Topcoat のセッションの札（トークン）を、どこに置いて、どう読むかを決めるトレイト（決まりごと）。この資料では、HTTP でもログインが続くように、自分で `HttpCookieTokenStore` を作って、この決まりを守らせます。	04 6. アプリを作る
`TokenStoreFuture`	`TokenStore` のメソッドが返す、「あとで答えが出る仕事（Future）」の型。	04 6. アプリを作る
`.sessions()`	ルーターに、セッション（ログインの状態を覚える仕組み）を登録するメソッド。`.sessions(SessionConfig::default())`。先に `.cookies()` も登録します。	04 6. アプリを作る
`.token_store()`	セッションの札（トークン）を、どこに置くかを決めるメソッド。ここでは、HTTP のページでもログインが続くように、自分で用意した `HttpCookieTokenStore`（クッキーに置く係）を渡しています。	04 6. アプリを作る
`.lifetime()`	セッションの長さ（何も操作しないと切れるまでの時間）を決めるメソッド。`.lifetime(IDLE)`（`IDLE` は 60 分）。	04 6. アプリを作る
`SessionConfig::builder()`	Topcoat のセッションの設定を、少しずつ決めて組み立て始める関数。`.token_store(…)`・`.lifetime(…)` をつなげて、最後に `.build()` で出来上がりです。	04 6. アプリを作る
`SessionConfig`	Topcoat のセッションの設定（クッキーの名前・期限など）の型。	04 6. アプリを作る

JavaScript・TypeScript

コード	意味と使い方	初めて出てくる所
`import`	ほかのファイルや部品から、関数などを読み込む書き方。`import { rmSync } from "node:fs";` は、Node.js のファイルの道具から `rmSync` を使えるようにします。	05 6. アプリの作り
`from`	`import { 名前 } from "場所";` の一部で、「どこから読み込むか」を書きます。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`default`	`export default` は、そのファイルから「いちばんの物」を 1 つ外に出す書き方。読み込む側は `import 名前 from "./ファイル"` と、好きな名前で受け取れます。	05 6. アプリの作り
`//`	JavaScript・TypeScript では、`//` から行の終わりまでがコメントです。	05 6. アプリの作り
`defineConfig()`	`defineConfig({ … })` は、設定ファイル（Vite の `vite.config` や Playwright の `playwright.config`）で、設定を書くための関数です。書き間違いを、エディターが教えてくれるようになります。	05 6. アプリの作り
`const`	JavaScript・TypeScript で、変えない値に名前を付ける書き方。`const db = …;`。あとで別の値を入れようとするとエラーになります。	05 6. アプリの作り
`return`	JavaScript・TypeScript で、関数の結果を返して、関数を終える命令。	05 6. アプリの作り
`async`	JavaScript・TypeScript で、中で `await` を使える関数にする印。結果は `Promise`（あとで届く約束）で返ります。	05 6. アプリの作り
`await`	JavaScript・TypeScript で、時間のかかる処理（`Promise`）が終わるまで待って、結果を受け取る書き方。`async` の関数の中で使います。	05 6. アプリの作り
`if`	JavaScript・TypeScript の条件。`if (条件) { … } else { … }`。条件は `( )` で囲みます。	05 6. アプリの作り
`new`	クラスから、新しい物を作る書き方。`new DatabaseSync("ba.sqlite")`・`new Date()`・`new Error("…")` のように使います。	05 6. アプリの作り
`.json()`	`fetch` の答え（`Response`）の本文を、JSON として読んで、オブジェクトにするメソッド。`await res.json()`。	05 6. アプリの作り
`=>`	アロー関数（短く書ける関数）の矢印。`(path) => `…${path}`` は「`path` を受け取って文字列を返す関数」。	05 6. アプリの作り
`/…/`	JavaScript の `/…/` は、正規表現（文字の形を表す書き方）をそのまま書く形です。`/\d+/` は「数字が 1 つ以上」。`.match(/…/)`・`.test(…)`・`.split(/…/)` などで使います。記号ごとの意味は、この一覧の「正規表現」の表にあります。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`${…}`	`` `…` ``（バッククォートで囲んだ文字。テンプレート文字列）の中の `${ }` には、値や式を入れられます。`` `エラー ${res.status}` `` は、`res.status` が 404 なら `エラー 404` になります。改行も、そのまま書けます。	05 6. アプリの作り
`!==`	`a !== b` は、「a と b が同じではない」（型も含めて比べる）です。`===`（同じ）の反対です。	05 6. アプリの作り
`||`	`a || b` は、「a か b のどちらか」（または）です。`el.innerText || el.getAttribute("name") || ""` のように、「左が空なら右を使う」という使い方もします（`0` や `""` も「空」とみなされるので、`null`・`undefined` だけを見たい時は `??` を使います）。	05 6. アプリの作り
`!`	`!res.ok` のように前に付けた `!` は、「〜ではない」です。`if (!res.ok)` は「うまくいかなかったら」。	05 6. アプリの作り
`条件 ? A : B`	`条件 ? A : B` は、「条件が合えば A、合わなければ B」になる式です（三項演算子）。`user.isAdmin ? "管理者" : "一般"`。`if` とちがい、値を返すので、そのまま変数や表示に使えます。	05 6. アプリの作り
`() =>`	`() => { … }` は、名前の無い関数（アロー関数）の書き方です。`( )` の中が引数、`=>` の右が中身です。`(e) => setKeyword(e.target.value)` のように、1 つの式なら `{ }` を省けます。ほかの関数に「あとで呼んでもらう処理」を渡す時によく使います。	05 6. アプリの作り
`Error`	`new Error("説明")` は、エラーを作ります。`throw new Error(…)` でエラーを起こし、`try { … } catch (err) { … }` で受け止めます。`err.message` で説明の文を読めます。	05 6. アプリの作り
`decodeURIComponent()`	`decodeURIComponent("%E5%B1%B1")` は、URL やクッキーの中で `%XX` の形にした文字を、元の文字（山）に戻します。反対は `encodeURIComponent()` です。	05 6. アプリの作り
`fetch()`	`fetch(URL)` は、サーバーに HTTP の要求を送る関数です。`await fetch(…)` で答えを待ち、`res.ok`（うまくいったか）・`res.status`（状態コード）・`await res.json()`（JSON の中身）を読みます。	05 6. アプリの作り
`.cookie`	`document.cookie` は、そのページのクッキーを、`名前=値; 名前=値` の形の文字で読みます（`HttpOnly` のクッキーは読めません）。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`.ok`	`fetch` の答えの `.ok` は、状態コードが 200〜299（うまくいった）なら `true` です。	05 6. アプリの作り

TypeScript

コード	意味と使い方	初めて出てくる所
`export`	「このファイルの外から使ってよい」という印。`export type RequestStatus = …` と書くと、ほかのファイルで `import` して使えます。	05 6. アプリの作り
`function`	関数（ひとまとまりの処理）を作る書き方。`function greet(name: string): string { … }` は、「文字列の `name` を受け取って、文字列を返す関数 `greet` 」です。	05 6. アプリの作り
`undefined`	「まだ値が入っていない」ことを表す値。`?` の付いた無くてもよい項目が無い時や、配列の範囲の外を読んだ時に出てきます。	05 6. アプリの作り
`catch`	`try` の中で失敗した時に動く所。`catch (e)` の `e` にエラーが入ります（何が来るか分からないので、使う前に確かめます）。	05 6. アプリの作り
`throw`	エラーを起こして、処理を止める書き方。`throw new Error("サーバーのエラー")`。`try … catch` で受け止められます。	05 6. アプリの作り
`.match()`	文字列が正規表現に合う所を探すメソッド。見つかると、その情報（かっこの中の部分など）を配列で、見つからないと `null` を返します。	05 6. アプリの作り
`.stringify()`	`JSON.stringify(値)` は、オブジェクトや配列を、JSON の文字列にする関数です。保存したり、サーバーへ送ったりする時に使います。	05 6. アプリの作り
`.catch()`	`Promise` が失敗した時の代わりの処理を登録するメソッド。`res.json().catch(() => ({}))` は「JSON として読めなければ空のオブジェクト」。	05 6. アプリの作り
`...`	スプレッド構文。`[...requests]` は配列の中身を広げて、新しい配列（写し）を作ります。`{ ...user, isAdmin: true }` は、オブジェクトを写して一部だけ変えます。	05 6. アプリの作り

React

コード	意味と使い方	初めて出てくる所
`react()`	`react()` は、Vite に、React（JSX）のファイルを扱えるようにする部品（プラグイン）です。	05 6. アプリの作り

Python

コード	意味と使い方	初めて出てくる所
`import`	ほかのモジュール（部品）を読み込む書き方。`import re` で正規表現の道具、`from pathlib import Path` で `Path` だけを使えるようにします。	05 6. アプリの作り
`from`	`from パッケージ import 名前` の一部で、「どこから読み込むか」を書きます。`from pathlib import Path`。	05 6. アプリの作り
`def`	関数を作る書き方。`def greet(name: str) -> str:` の次の行から、**字下げした所が関数の中身**です。Python は `{ }` の代わりに字下げで「どこまでが中身か」を表すので、字下げをそろえることがとても大事です。	05 6. アプリの作り
`None`	Python の「値が無い」ことを表す値。調べる時は `x is None` と書きます。	05 6. アプリの作り
`if`	Python の条件。`if 条件:` の次の行から、字下げした所が中身です。`a if 条件 else b` の形で、1 行の中で値を選ぶこともできます。	05 6. アプリの作り
`is`	「同じ物か」を調べる書き方。`if value is None:` は「値が無ければ」。`None` と比べる時は `==` でなく `is` を使います。	05 6. アプリの作り
`raise`	エラー（例外）を起こして、処理を止める書き方。`raise TransitionError("…")`。呼んだ側は `try … except` で受け止められます。	05 6. アプリの作り
`return`	Python の関数の結果を返して、関数を終える命令。	05 6. アプリの作り
`for`	Python のくり返し。`for x in 並び:` の次の行から、字下げした所が中身です。`[式 for x in 並び]` の形（内包表記）で、新しいリストを 1 行で作ることもできます。	05 6. アプリの作り
`in`	`for x in 並び`（並びから 1 つずつ）と、`値 in 並び`（含まれているか）の 2 つの使い方があります。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`True`	Python の「正しい（はい）」の値。反対は `False`。頭は大文字です（`true` ではありません）。	05 6. アプリの作り
`False`	Python の「正しくない（いいえ）」の値。	05 6. アプリの作り
`.resolve()`	`Path(__file__).resolve()` は、ファイルの場所を、`C:\` から始まる絶対の場所にするメソッド。`.parent` で 1 つ上のフォルダになります。	05 6. アプリの作り
`.get()`	辞書（`dict`）から、キーで値を取り出すメソッド。`d.get("port", 8080)` は、キーが無ければ 2 つ目の値（無ければ `None`）を返すので、`d["port"]` と違ってエラーになりません。	05 6. アプリの作り
`.lower()`	英字を小文字にした文字列を返すメソッド。`"True".lower() == "true"` のように、大文字小文字をそろえて比べます。	05 6. アプリの作り
`.split()`	Python で、文字列を分けてリストにするメソッド。`"a b c".split()` のように何も渡さないと、空白（いくつ続いても）で分けます。	05 6. アプリの作り
`.match()`	`re.match(パターン, 文字列)` は、文字列の**先頭から**正規表現に合うかを調べる関数です。合えば、見つかった物が返ります（合わなければ `None`）。	05 6. アプリの作り
`str()`	値を文字列に変える関数。`str(8080)` は `"8080"`。数と文字列は `==` で比べても等しくならないので、そろえる時に使います。	05 6. アプリの作り
`int()`	文字列などを整数にする関数。`int("25")` は 25。数でない文字だと `ValueError` です。	05 6. アプリの作り
`#`	Python では、`#` から行の終わりまでがコメントです。	05 6. アプリの作り
`"""…"""`	`"""…"""`（`"` を 3 つ）で囲んだ文字は、何行にもまたがって書けます。関数・クラス・ファイルのいちばん最初に書いた物は、その説明書（docstring）になり、`help()` やエディターで読めます。	05 6. アプリの作り
`f"…{値}…"`	`f"…"`（頭に `f` を付けた文字）の中の `{ }` には、変数や式を書けて、その値が入ります（f 文字列）。`f"環境変数 {name} がありません"`。`{ }` の中の `:` の後ろには、`{amount:,}` のように書式も書けます。	05 6. アプリの作り
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。正規表現の `r"\d{6}"` のように、`\` の多い文字をそのまま書けます。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`[式 for 名前 in 並び]`	`[f"http://{host}" for host in ALLOWED_HOSTS]` は、「並びの 1 つずつから、新しい値を作って、並び（リスト）にする」書き方です（リスト内包表記）。後ろに `if 条件` を付けると、条件に合う物だけになります。	05 6. アプリの作り
`==`	`a == b` は、「a と b が等しいか」です。`=`（1 つ）は「値を入れる」です。	05 6. アプリの作り
`名前=値`	`open(path, encoding="utf-8")` の `encoding="utf-8"` のように、`名前=値` で渡す引数は「キーワード引数」です。順番に関係なく、名前で渡せます。	05 6. アプリの作り
`__file__`	`__file__` は、今のファイルの場所が入った、特別な変数です。`Path(__file__).resolve().parent` で、このファイルが置いてあるフォルダが分かります。	05 6. アプリの作り
`Path`	`Path`（`pathlib`）は、ファイルやフォルダの場所を表す型です。`Path("memo.txt").exists()`・`.read_text()`・`/` でのつなぎ（`BASE_DIR / "media"`）ができます。	05 6. アプリの作り
`RuntimeError`	`RuntimeError` は、ほかの種類に当てはまらない、動いている時のエラーです。`raise RuntimeError("…")` で、説明を付けて止めます。	05 6. アプリの作り
`.parent`	`Path` の `.parent` は、1 つ上のフォルダです。`.parent.parent` で 2 つ上です。	05 6. アプリの作り
`.environ`	`os.environ` は、環境変数の入れ物（辞書のような物）です。`os.environ.get("名前", 無い時の値)` で読みます。	05 6. アプリの作り
`class`	クラス（データと、それを扱う関数をまとめた設計図）を作る書き方。`class Request:` の次から、字下げした所が中身です。	05 6. アプリの作り
`or`	「または」。`a or b` は、`a` が空（`None`・`""`・0 など）なら `b` を使います。`(row.get("番号") or "").strip()` は「無ければ空の文字列にしてから空白を取る」。	05 6. アプリの作り
`.loads()`	文字列から読み取るメソッド。`tomllib.loads(文字列)` は TOML、`json.loads(文字列)` は JSON を、Python の辞書にします。	05 6. アプリの作り
`len()`	長さ（中身の数）を返す関数。`len("abc")` は 3、`len([1, 2])` は 2、`len(sys.argv)` は「渡された引数の数＋1」です。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`b"…"`	`b"…"` は、文字ではなく「バイトの並び」（ `bytes` ）を作る書き方です。`b"\xff\xd8\xff"` は、JPEG の画像のファイルの、最初の 3 バイトです。	05 6. アプリの作り

Django

コード	意味と使い方	初めて出てくる所
`.get_username()`	Django の利用者から、ログインに使う名前（この資料では社員番号）を返すメソッド。	05 6. アプリの作り
`method_decorator()`	関数用の印（デコレーター）を、クラスの画面（ビュー）のメソッドにも付けられるようにする物。	05 6. アプリの作り
`authenticate()`	名前とパスワードが正しいかを確かめる Django の関数。正しければ利用者（ `User` ）を、違えば `None` を返します。	05 6. アプリの作り
`login()`	確かめた利用者を、ログインした状態にする Django の関数（セッションを始める）。`login(request, user)` 。	05 6. アプリの作り
`logout()`	ログアウトさせる Django の関数（セッションを消す）。`logout(request)` 。	05 6. アプリの作り
`@method_decorator`	`@method_decorator(ensure_csrf_cookie, name="dispatch")` をクラスの上に付けて、そのクラスの画面に印を付けます。	05 6. アプリの作り
`@require_POST`	「この画面は POST だけを受け付ける」という印（デコレーター）。GET などで来たら `405` を返します。ログアウトのように、状態を変える物に付けます。	05 6. アプリの作り
`JsonResponse`	Django の `JsonResponse({"status": "ok"})` は、辞書を JSON にして返す答えです。	05 6. アプリの作り
`TemplateView`	Django の `TemplateView` は、決まったテンプレート（HTML）を表示するだけのビューです。`template_name = "index.html"` を書くだけで使えます。	05 6. アプリの作り
`.body`	Django の `request.body` は、届いた要求の中身（バイトのまま）です。JSON なら `json.loads(request.body)` で読みます。	05 6. アプリの作り

コード	意味と使い方	初めて出てくる所
`.user`	Django の `request.user` は、今ログインしている人（していなければ「だれでもない人」）です。`request.user.is_authenticated` で、ログインしているかが分かります。	05 6. アプリの作り
`.META`	Django の `request.META` は、要求の見出しやサーバーの情報が入った辞書です。`request.META.get("REMOTE_ADDR")` で、送ってきた所のアドレスが分かります。	05 6. アプリの作り
`.as_view()`	クラスで作った画面（ビュー）を、URL の決まりに登録できる形にするメソッド。	05 6. アプリの作り
`path()`	URL と、それに答える画面（ビュー）を結び付ける関数。`path("requests/<int:pk>/", views.detail)` の `<int:pk>` は、URL の中の数を `pk` という名前で受け取ります。	05 6. アプリの作り
`re_path()`	正規表現で URL の決まりを書く関数。`re_path(r"^(?!api/|admin/).*$", …)` は「`api/`・`admin/` で始まらない URL は全部」（React の画面を返す）。	05 6. アプリの作り
`.urls`	`admin.site.urls` は、Django の管理画面の URL の一覧です。`path("admin/", admin.site.urls)` で、`/admin/` に管理画面を出します。	05 6. アプリの作り
`.site`	`admin.site` は、Django の管理画面そのものです。`admin.site.register(モデル)` で、モデルを管理画面に出します。	05 6. アプリの作り

Django REST framework

コード	意味と使い方	初めて出てくる所
`api_view()`	関数を、DRF の API の画面にする物。`@api_view(["GET", "POST"])` のように使います。	05 6. アプリの作り
`@api_view`	`@api_view(["GET", "POST"])` を関数の上に付けて、API にします。決めたメソッド以外は 405 で断られます。	05 6. アプリの作り
`Response`	DRF の `Response(data)` は、API の答えです。辞書や並びを、JSON にして返します。	05 6. アプリの作り